

Master's thesis
Master's Programme in Computer Science

Hiệu năng, an toàn và đánh đổi kỹ thuật của Rust và C++ trong hệ thống điều khiển RAN của Nokia

Ha Vien

July 25, 2026

Supervisor(s): Christoph Luneburg, Petteri Kotimaa

Examiner(s):

University of Helsinki
Faculty of Science

P. O. Box 68 (Pietari Kalmin katu 5)
00014 University of Helsinki

HELSINGIN YLIOPISTO — HELSINGFORS UNIVERSITET — UNIVERSITY OF HELSINKI

Tiedekunta — Fakultet — Faculty	Koulutusohjelma — Utbildningsprogram — Degree programme
Faculty of Science	Master's Programme in Computer Science

Tekijä — Författare — Author

Ha Vien

Työn nimi — Arbetets titel — Title

Hiệu năng, an toàn và đánh đổi kỹ thuật của Rust và C++ trong hệ thống điều khiển RAN của Nokia

Työn laji — Arbetets art — Level	Aika — Datum — Month and year	Sivumäärä — Sidantal — Number of pages
Master's thesis	July 25, 2026	69

Tiivistelmä — Referat — Abstract

Bối cảnh. Phần mềm mặt điều khiển của Mạng truy nhập vô tuyến (RAN) — ví dụ tầng L3 của một gNB — chủ yếu được viết bằng C++ vì hiệu năng, nhưng C++ mang theo những rủi ro đã được biết rõ về an toàn bộ nhớ và tương tranh. Rust cung cấp hiệu năng tương đương với các bảo đảm an toàn được kiểm tra tại thời điểm biên dịch, song mức độ phù hợp của nó cho các mặt điều khiển viễn thông nhạy độ trễ vẫn chưa được chứng minh. Công trình này được thực hiện tại Nokia trên một phân hệ được mô hình hoá theo một thành phần gNB-CU-CP thực tế của Nokia.

Mục tiêu. Luận văn so sánh C++ hiện đại và Rust trên một dịch vụ Quản lý Ngữ cảnh UE (UE Context Management). Vì thông lượng trung bình tương tự nhau đã được biết trước, luận văn nhắm tới bốn khía cạnh ít được khảo sát hơn: độ trễ đuôi và độ dao động (jitter) khi quá tải; những lớp lỗi C++ nào được Rust ngăn chặn tại thời điểm biên dịch; chi phí phụ trội của mô hình an toàn của Rust; và chi phí áp dụng tăng dần trong một mã nguồn C++ sẵn có.

Phương pháp. Trước hết, một đặc tả độc lập ngôn ngữ được cố định (đóng băng); sau đó hai bản hiện thực tương đương về chức năng được xây dựng bằng C++ và Rust với cùng thuật toán, mô hình tương tranh, bộ cấp phát bộ nhớ và cấu hình biên dịch. Một khối lượng công việc tổng hợp, được biện minh theo đặc tả, được quét tăng dần đến mức quá tải; cả hai bản đều vượt qua cùng một bộ kiểm thử tương đương (parity), và phần đánh giá bổ sung thêm một nghiên cứu tiêm lỗi và một nguyên mẫu khả năng tương tác FFI nhỏ.

Kết quả. Kết quả về an toàn và di trú là kết quả cuối cùng. Mỗi một trong năm lớp lỗi C++ được tiêm vào (sử dụng sau khi giải phóng, giải phóng hai lần, vô hiệu hoá iterator, tranh chấp dữ liệu, biến đổi trạng thái chia sẻ không được bảo vệ) đều bị ngăn chặn tại thời điểm biên dịch trong Rust an toàn, trong khi ở C++ mỗi lỗi đều biên dịch được và chỉ bị phát hiện, nếu có, tại thời điểm chạy và chỉ khi dùng đúng bộ dò lỗi (sanitiser). Nguyên mẫu di trú FFI cho thấy một đường ghép (seam) nhỏ giúp lõi Rust không cần `unsafe` và không thêm chi phí đo được nào cho mỗi lời gọi. Các phép đo hiệu năng sơ bộ cho thấy hai ngôn ngữ bám sát nhau đến điểm bão hoà, với khác biệt chỉ giới hạn ở phần đuôi xa; các số liệu độ trễ đạt chuẩn testbed là bước còn lại.

Kết luận. Luận văn cung cấp một khung đo hiệu năng có thể tái lập, một cặp bản hiện thực đã kiểm thử tương đương, một phân loại (taxonomy) các lớp lỗi, và một chi phí di trú được định lượng; đồng thời lập luận rằng đối với lớp phân hệ này, rào cản áp dụng Rust là công sức kỹ thuật và đường cong học tập chứ không phải hiệu năng lúc chạy.

ACM Computing Classification System (CCS):
Software and its engineering → Software notations and tools → General programming languages
Computer systems organization → Real-time systems → Real-time system specification
Networks → Network types → Mobile networks

Avainsanat — Nyckelord — Keywords
Rust, C++, điều khiển RAN, trễ đuôi, an toàn bộ nhớ, tiêm lỗi, FFI

Säilytyspaikka — Förvaringsställe — Where deposited
Thư viện Đại học Helsinki

Muita tietoja — Övriga uppgifter — Additional information
Software study track

Mục lục

1 Giới thiệu **1**

1.1 Thuật ngữ và từ viết tắt chính 2
1.2 Động lực và bối cảnh công nghiệp 2
1.3 Thách thức trong kỹ nghệ phần mềm mặt điều khiển RAN 5
1.4 Phát biểu bài toán 5
1.5 Các câu hỏi nghiên cứu 6
1.6 Cách tiếp cận nghiên cứu tổng quan 6
1.7 Đóng góp, kết quả và giới hạn 7
1.8 Ý nghĩa và đối tượng hưởng lợi 8
1.9 Bố cục luận văn 8

2 Nền tảng: Các ngôn ngữ hệ thống cho phần mềm nhạy độ trễ **11**

2.1 Các hệ thống mặt điều khiển RAN và gNB-CU-CP 11
2.2 Độ trễ, thông lượng và tính dự đoán được trong phần mềm thời gian thực 12
2.3 C++ cho các hệ thống nhạy độ trễ 14
2.4 Mô hình quyền sở hữu và an toàn của Rust 15
2.5 Bức tranh toàn cảnh các ngôn ngữ hệ thống 17
2.6 Các mô hình tương tranh cho phần mềm mặt điều khiển 18
2.7 Khả năng tương tác giữa C++ và Rust 20
2.8 Khoảng trống nghiên cứu 20

3 Phương pháp nghiên cứu **21**

3.1 Cách tiếp cận nghiên cứu 21
3.2 Một thí nghiệm kiến tạo, so sánh 21
3.3 Các phương pháp cho bốn câu hỏi nghiên cứu 23
3.3.1 Đo hiệu năng dưới một phép quét tải (RQ1, RQ3) 23
3.3.2 Tiêm lỗi có kiểm soát (RQ2) 24
3.3.3 Nghiên cứu tình huống di trú qua giao diện hàm ngoại (RQ4) . 24
3.4 Tính công bằng như một chiến lược biến-được-kiểm-soát 25

3.5 Xử lý thống kê các phép đo 26
3.6 Tính tái lập và các sản phẩm nghiên cứu 26
3.7 Các cân nhắc về đạo đức và bảo vệ dữ liệu 27

4 Bối cảnh bài toán: Quản lý Ngữ cảnh UE và tính dự đoán được 29
4.1 Phân hệ Quản lý Ngữ cảnh UE 29
4.2 Các lớp lỗi an toàn bộ nhớ và tương tranh 30
4.3 Độ trễ đuôi và tính dự đoán được trong RAN 32

5 Mục tiêu 35
5.1 Những thiếu sót trong các phép so sánh hiện có 35
5.2 Các mục tiêu giải pháp 35
5.3 Các giả thuyết 36
5.4 Các tiêu chí thành công 36
5.5 Phạm vi và ranh giới 37

6 Thiết kế và Hiện thực 39
6.1 Đặc tả chung: đường cơ sở công bằng 39
6.2 Mô hình khái niệm và thiết kế tương tranh 39
6.3 Các lựa chọn kiến trúc và ngăn xếp công nghệ 40
6.4 Chi tiết hiện thực 41
6.5 API công khai và ngữ nghĩa thao tác 42
6.6 Vòng đời yêu cầu và sự chuyển giao hoàn tất 43
6.7 Kiểm chứng tương đương 43

7 Thiết lập thực nghiệm và Đánh giá 47
7.1 Cấu hình testbed 47
7.2 Khối lượng công việc và các phép đo đánh giá 47
7.3 Thu thập dữ liệu và các thước đo 48
7.4 Dữ liệu đánh giá 48
7.5 Các kết quả đánh giá 49
7.5.1 Hiệu năng (RQ1, RQ3) 49
7.5.2 An toàn qua tiêm lỗi (RQ2) 51
7.5.3 Di trú (RQ4) 54

8 Thảo luận 57
8.1 Tóm tắt các phát hiện 57
8.2 Thảo luận về các câu hỏi nghiên cứu 57
8.3 Công trình liên quan 59

8.4 Các mối đe doạ đối với tính hợp lệ và biện pháp giảm thiểu 61

9 Kết luận và Công việc tương lai **63**

Tài liệu tham khảo **65**

Phụ lục A Ví dụ tiêm lỗi **69**

1. Giới thiệu

Mạng di động đang trải qua một quá trình phần mềm hoá sâu sắc. Những chức năng từng được cung cấp bởi phần cứng chuyên dụng, tích hợp dọc, nay đang được hiện thực lại thành phần mềm chạy trên các máy chủ phổ thông, và Mạng truy nhập vô tuyến (RAN) — vốn từ lâu là phần chuyên biệt nhất của hệ thống — không còn là ngoại lệ. Trong kiến trúc 5G NG-RAN, trạm gốc (gNB) được phân rã thành một đơn vị trung tâm (central unit) và một hoặc nhiều đơn vị phân tán (distributed unit), và đơn vị trung tâm lại được tách tiếp thành các chức năng mặt điều khiển và mặt người dùng [1, 3]. Các sáng kiến Open-RAN đẩy sự phân rã này đi xa hơn, biến RAN thành phần mềm ảo hoá, đa nhà cung cấp, kết nối bằng các giao diện mở [27]. Mặt điều khiển thu được, tức gNB-CU-CP, là một hệ thống phần mềm lớn, có trạng thái, tương tranh cao: nó quản lý vòng đời của mọi thiết bị đang kết nối, điều khiển các thủ tục báo hiệu RRC [4], và điều phối tính di động cùng các cuộc chuyển giao (handover), tất cả dưới những ngân sách độ trễ ngặt nghèo và ở tốc độ sự kiện rất cao: một trạm bận có thể thực hiện tới hàng nghìn thủ tục quản lý kết nối và di động mỗi giây, mỗi thủ tục bị giới hạn bởi một ngân sách đo bằng mili-giây [5], và một mặt điều khiển đã triển khai phải điều phối hàng triệu thiết bị kết nối đồng thời. Khi RAN chuyển sang các triển khai gốc-đám-mây (cloud-native), phân rã, thì tính đúng đắn, độ tin cậy và hành vi định thời của phần mềm này ngày càng quyết định chất lượng của toàn mạng. Một sự cố ngừng dịch vụ hay một sự thoái lui về độ trễ đuôi ở đây không phải là một khiếm khuyết cục bộ; nó là một sự suy giảm cảm nhận được trên mọi thiết bị mà trạm phục vụ [12, 9].

Phần mềm mặt điều khiển này, hiện nay, chủ yếu được viết bằng C++. Lựa chọn đó là hợp lý: C++ cung cấp khả năng kiểm soát mức thấp, công cụ trưởng thành và hiệu năng mà các hệ thống nhạy độ trễ đòi hỏi. Nhưng hiệu năng đó đi kèm một cái giá đã được biết rõ. C++ đặt gánh nặng quản lý bộ nhớ và tương tranh lên vai lập trình viên, và hàng thập kỷ kinh nghiệm cho thấy gánh nặng này thường xuyên bị xử lý sai, tạo ra các lỗi sử-dụng-sau-khi-giải-phóng, giải-phóng-hai-lần, tranh chấp dữ liệu, và các khiếm khuyết liên quan — những lỗi tốn kém để tìm, nguy hiểm trong môi trường sản xuất, và chịu trách nhiệm cho một phần lớn các lỗ hổng bảo mật trong thực tiễn công nghiệp [22, 32]. Rust nổi lên như ngôn ngữ hệ

thống chính thống đầu tiên hứa hẹn thoát khỏi sự đánh đổi này: an toàn bộ nhớ và không có tranh chấp dữ liệu được bảo đảm tại thời điểm biên dịch, với hiệu năng nhắm tới việc cạnh tranh được với C++ [20, 18]. Liệu lời hứa đó có đúng cho một mặt điều khiển cấp viễn thông, nhạy độ trễ, nơi tính dự đoán được khi quá tải quan trọng ngang với tốc độ thuần tuý, thì vẫn chưa được hiểu rõ. Luận văn này, được thực hiện tại Nokia, giải quyết câu hỏi đó một cách trực tiếp: nó phát triển hai bản hiện thực tương đương về chức năng của một phân hệ RAN tiêu biểu — một dịch vụ Quản lý Ngữ cảnh UE được mô hình hoá theo một thành phần gNB-CU-CP thực tế của Nokia — bằng C++ hiện đại và bằng Rust, rồi so sánh chúng dưới một khối lượng công việc báo hiệu tổng hợp thực tế.

1.1 Thuật ngữ và từ viết tắt chính

Đối với độc giả ngoài lĩnh vực viễn thông, Bảng 1.1 cố định các từ viết tắt và thuật ngữ được dùng xuyên suốt luận văn. Ba trong số đó lặp lại liên tục: *gNB-CU-CP* là phần mềm mặt điều khiển mà công trình này bàn về; *Quản lý Ngữ cảnh UE (UECM)* là phân hệ cụ thể được hiện thực lại và đo đạc; và *độ trễ đuôi* (các phân vị p99/p99.9 thay vì giá trị trung bình) là thuộc tính mà, hơn cả tốc độ trung bình, quyết định liệu một hệ thống như vậy có đáp ứng được các hạn chót của nó hay không [12, 9].

1.2 Động lực và bối cảnh công nghiệp

Hai xu hướng khiến phép so sánh này trở nên đúng lúc. Thứ nhất là sự tiến hoá gốc-đám-mây của chính RAN. Việc đưa các chức năng mặt điều khiển lên các máy chủ đa dụng làm *tăng* chứ không giảm mức đòi hỏi về chất lượng phần mềm: phần mềm tồn tại lâu hơn, tiến hoá liên tục, chạy ở mức tương tranh cao hơn, và phải đáp ứng các hạn chót mà không có sự tất định (determinism) mà phần cứng chuyên dụng từng cung cấp. Sự chuyển dịch này được thúc đẩy nhanh hơn bởi sự phân rã Open-RAN, vốn biến trạm gốc thành phần mềm ảo hoá, đa nhà cung cấp, kết nối bằng các giao diện mở [27], khiến cho mặt điều khiển mà một nhà cung cấp bàn giao ngày càng cạnh tranh dựa trên chất lượng kỹ nghệ phần mềm chứ không phải trên phần cứng chuyên biệt. Trong bối cảnh này, một sự thoái lui về độ trễ hay một khiếm khuyết hỏng bộ nhớ không phải là một lỗi cục bộ mà là một rủi ro về độ tin cậy và bảo mật trên phạm vi toàn mạng. Xu hướng thứ hai là sự chuyển dịch toàn ngành hướng tới các ngôn ngữ an toàn bộ nhớ. Các tổ chức phần mềm lớn báo cáo rằng phần lớn các lỗ hổng bảo mật nghiêm trọng của họ bắt nguồn từ các lỗi an toàn bộ nhớ, và rằng việc áp dụng ngôn ngữ an toàn bộ nhớ giảm chúng một

Bảng 1.1: Các từ viết tắt và thuật ngữ chính dùng xuyên suốt luận văn.

Thuật ngữ	Ý nghĩa
RAN	Radio Access Network — Mạng truy nhập vô tuyến; phần mạng di động kết nối thiết bị với lõi mạng
NG-RAN	Next-Generation RAN, kiến trúc mạng truy nhập vô tuyến của 5G
gNB	Trạm gốc 5G (“next-generation NodeB”)
gNB-CU / DU	Central Unit / Distributed Unit; hai nửa được phân rã của một gNB
gNB-CU-CP	Phần mặt điều khiển của đơn vị trung tâm; lớp hệ thống được nghiên cứu ở đây
UE	User Equipment — Thiết bị người dùng; thiết bị đầu cuối (điện thoại, modem) gắn vào mạng
UECM	UE Context Management — Quản lý Ngữ cảnh UE; phân hệ quản lý trạng thái theo từng UE được hiện thực lại ở đây
RRC	Radio Resource Control; giao thức báo hiệu mặt điều khiển giữa UE và gNB
DRB	Data Radio Bearer; kênh dữ liệu mặt người dùng theo từng UE được thiết lập trong một phiên
Mặt điều khiển	Đường báo hiệu và quản lý, đối lập với đường dữ liệu mặt người dùng
Độ trễ đuôi	Độ trễ ở phân vị cao (p99, p99.9), thước đo định thời hướng trường hợp xấu nhất
FFI	Foreign Function Interface; gọi Rust từ C++ qua một C ABI
HDR histogram	Biểu đồ dải-động-cao, dùng để ghi nhận độ trễ chính xác

cách đo được [22, 32, 29]. Mối lo ngại này đã dâng lên tới cấp chính sách quốc gia: các cơ quan an ninh mạng nay kêu gọi các nhà sản xuất phần mềm công bố các "lộ trình an toàn bộ nhớ" và di trú khỏi các ngôn ngữ không an toàn bộ nhớ như C và C++ [11, 25]. Riêng Rust đã chuyển từ một ngôn ngữ ngách vào trong các nhân hệ điều hành, hạ tầng đám mây, và ngày càng nhiều vào viễn thông; các hypervisor sản xuất như AWS Firecracker, nền tảng cho dịch vụ serverless của họ, đã được viết bằng Rust [6]. Tuy vậy, cơ sở bằng chứng cho sự phù hợp của Rust phần lớn dựa trên phần mềm đa dụng và trên các phép đo hiệu năng trường hợp trung bình. Câu hỏi cụ thể mà một nhà cung cấp viễn thông phải trả lời — liệu Rust có đáp ứng được các yêu cầu về độ trễ đuôi và tính dự đoán được của một mặt điều khiển RAN, liệu nó có thực sự ngăn chặn các lớp lỗi đang gây hại cho mã C++ hiện có, và chi phí áp dụng nó tăng dần bên trong một mã nguồn C++ lớn là bao nhiêu — vẫn chưa được trả lời bằng bằng chứng trực tiếp, có kiểm soát. Cung cấp bằng chứng như vậy cho một phân hệ tiêu biểu chính là động lực của công trình này.

Bối cảnh công nghiệp. Luận văn này được thực hiện tại Nokia, một trong số ít nhà cung cấp xây dựng, bảo trì và phát triển phần mềm RAN sản xuất một cách toàn trình (end-to-end), và phân hệ mà nó nghiên cứu — Quản lý Ngữ cảnh UE (UECM) — được mô hình hoá theo một thành phần mặt điều khiển gNB-CU-CP thực tế chứ không phải một ví dụ sách giáo khoa. gNB-CU-CP không phải là một mô-đun ngoại vi: trong kiến trúc phân rã của 3GPP, nó là nửa mặt điều khiển của đơn vị trung tâm, chứa RRC và quản lý vòng đời ngữ cảnh UE trong khi uỷ thác việc xử lý mang (bearer) mặt người dùng cho CU-UP qua giao diện E1 [1, 3, 2, 4], tất cả dưới các ngân sách thủ tục cỡ mili-giây mà hệ thống 5G đặt lên báo hiệu mặt điều khiển [5]. Phần mềm thuộc lớp này được bàn giao, tiến hoá và vận hành trên các mạng nhà khai thác đang hoạt động, nên một quyết định lựa chọn ngôn ngữ ở đây là một mối quan tâm sản xuất cụ thể chứ không phải giả định, và đó chính là điều làm cho một phép so sánh có kiểm soát trên một phân hệ tiêu biểu trở nên có giá trị đối với một nhà cung cấp. UECM là một lựa chọn có chủ đích làm đối tượng nghiên cứu trong bối cảnh đó: nó đủ nhỏ để hiện thực lại một cách trung thực và để lập luận đầy đủ, nhưng lại thể hiện đúng những thuộc tính khiến mã mặt điều khiển RAN trở nên khó: trạng thái khả biến theo từng UE, một vòng đời thao tác nghiêm ngặt, mức tương tranh cao trên nhiều UE, và những kỳ vọng ngặt nghèo về độ trễ đuôi. Do đó, các kết luận rút ra từ nó có thể chuyển sang lớp rộng hơn gồm các dịch vụ mặt điều khiển có trạng thái, tương tranh, đồng thời vẫn tái lập được đối với độc giả bên ngoài Nokia vì bản hiện thực được điều khiển hoàn toàn bởi một đặc tả công khai, đã đóng băng, chứ không phải bởi mã độc quyền.

Một chế độ hỏng cụ thể. Để làm rõ mức độ hệ trọng, hãy xét một "cơn bão di động" trong đó nhiều UE chuyển giao cùng lúc. Chỉ một con trỏ bị xử lý sai — một ngữ cảnh UE bị giải phóng khi release trong khi một thông điệp báo hiệu đang trong luồng vẫn còn tham chiếu tới nó — sẽ tạo ra một lỗi sử-dụng-sau-khi-giải-phóng. Trong C++ điều này biên dịch sạch sẽ và có thể vượt qua kiểm thử thông thường, để rồi làm hỏng trạng thái của một UE không liên quan hoặc làm sập luồng xử lý đúng vào những điều kiện tương tranh cao mà hệ thống sinh ra để xử lý; cùng một tranh chấp tiềm ẩn đó cũng có thể biểu hiện thành một đột biến độ trễ đuôi khó lường thay vì một cú sập rõ ràng. Trong Rust an toàn, cùng thiết kế đó không biên dịch được: quyền sở hữu và mượn (ownership và borrowing) loại bỏ tham chiếu treo ngay tại thời điểm biên dịch. Chỉ một kịch bản này đã nắm bắt được lý do vì sao luận văn đo cả *tính an toàn* (những lỗi nào được ngăn chặn, và khi nào) lẫn *tính dự đoán được* (độ trễ đuôi khi quá tải) chứ không chỉ thông lượng trung bình.

1.3 Thách thức trong kỹ nghệ phần mềm mặt điều khiển RAN

Các hệ thống mặt điều khiển RAN hiện đại phải nhanh, dự đoán được, tương tranh cao và tồn tại lâu dài, và chúng gần như hoàn toàn được viết bằng C++. Hai thuộc tính của lĩnh vực này khiến lựa chọn ngôn ngữ trở nên hệ trọng. Thứ nhất, việc quản lý bộ nhớ và tương tranh thủ công trong C++ phơi bày phần mềm ra các khiếm khuyết như sử-dụng-sau-khi-giải-phóng, giải-phóng-hai-lần, tranh chấp dữ liệu, và vô hiệu hoá iterator — những lỗi im lặng tại thời điểm biên dịch và tốn kém để tìm và sửa ở quy mô viễn thông. Thứ hai, thứ quyết định liệu các hạn chót RAN có được đáp ứng hay không không phải là thông lượng trung bình mà là hành vi trường hợp xấu nhất và phần đuôi khi tải và quá tải — một thuộc tính dễ bị thoái lui và khó đo cho đúng. Rust hứa hẹn loại bỏ lớp khiếm khuyết thứ nhất tại thời điểm biên dịch trong khi tuyên bố hiệu năng cạnh tranh được ở khía cạnh thứ hai, nhưng tuyên bố đó phần lớn dựa trên các phép đo đa dụng chứ không phải trên các khối lượng công việc viễn thông có trạng thái, tương tranh, nhạy độ trễ. Mục kế tiếp biến các thách thức lĩnh vực này thành những câu hỏi cụ thể, chưa được trả lời mà luận văn này đặt ra để giải quyết.

1.4 Phát biểu bài toán

Gộp các mạch suy nghĩ này lại, bài toán cụ thể mà luận văn giải quyết là như sau. Các nhà cung cấp viễn thông đang chịu áp lực ngày càng tăng, cả về kỹ thuật lẫn

ngày càng mang tính pháp lý, để áp dụng các ngôn ngữ an toàn bộ nhớ, nhưng đối với mặt điều khiển RAN họ thiếu bằng chứng *trực tiếp, có kiểm soát* để dựa vào cho quyết định đó. Chưa được xác lập liệu một bản hiện thực Rust của một phân hệ mặt điều khiển có trạng thái, tương tranh có thể sánh với C++ hiện đại về *độ trễ đuôi và tính dự đoán được khi quá tải* (chứ không phải thông lượng trung bình) hay không, liệu các bảo đảm tại thời điểm biên dịch của Rust có thực sự loại bỏ các lớp lỗi cụ thể đang gây hại cho mã C++ hiện có hay không, và việc áp dụng tăng dần *tốn kém* bao nhiêu bên trong một mã nguồn C++ lớn. Bằng chứng sẵn có phần lớn là đa dụng và trường hợp trung bình [8, 28], để ngỏ ba câu hỏi này (tính dự đoán được, phạm vi lỗi được ngăn chặn, và chi phí di trú) cho một phân hệ tiêu biểu. Trả lời chúng dưới các điều kiện có kiểm soát, tái lập được chính là bài toán mà luận văn này đặt ra để giải.

1.5 Các câu hỏi nghiên cứu

Luận văn được tổ chức quanh bốn câu hỏi nghiên cứu, được sắp thứ tự để đặt tính dự đoán được và di trú lên trước hiệu năng trung bình thuần tuý:

RQ1 Rust và C++ hiện đại so sánh như thế nào về tính dự đoán được của độ trễ (giá trị trung bình, độ trễ đuôi p99, p99.9, và độ dao động) cho một khối lượng công việc mặt điều khiển RAN khi tải tăng dần và quá tải?

RQ2 Những lớp lỗi an toàn bộ nhớ và tương tranh xuất hiện trong C++ nào được loại bỏ tại thời điểm biên dịch trong bản hiện thực Rust, và những lỗi còn lại được phát hiện ở giai đoạn nào và với chi phí bao nhiêu?

RQ3 Chi phí phụ trội, nếu có, của mô hình an toàn của Rust (kiểm tra biên, các bản sao do quyền sở hữu, chi phí trừu tượng hoá) lên thông lượng, mức sử dụng CPU và hành vi bộ nhớ là bao nhiêu?

RQ4 Những hệ quả thực tiễn (công sức, khả năng tương tác FFI, và rủi ro) của việc áp dụng Rust tăng dần bên trong một hệ sinh thái phần mềm RAN C++ sẵn có là gì?

1.6 Cách tiếp cận nghiên cứu tổng quan

Bốn câu hỏi được trả lời một cách kiến tạo chứ không phải bằng tổng hợp tài liệu. Sản phẩm cốt lõi là một cặp bản hiện thực UECM tương đương về chức năng, một bằng C++ hiện đại và một bằng Rust, được xây dựng từ một đặc tả đã đóng băng

duy nhất, sao cho mọi khác biệt đo được đều quy được cho ngôn ngữ và các thành ngữ (idiom) của nó chứ không phải cho các thiết kế khác nhau. RQ1 và RQ3 được trả lời bằng cách điều khiển cả hai bản hiện thực với một khối lượng công việc báo hiệu tổng hợp, tất định, giống hệt nhau trong một phép quét tải từ tải nhẹ qua bão hoà, ghi nhận độ trễ bằng các biểu đồ dải-động-cao khớp nhau và mức sử dụng tài nguyên thông qua hệ điều hành, rồi so sánh chúng bằng các khoảng tin cậy phi tham số [16]. RQ2 được trả lời bằng tiêm lỗi có kiểm soát: một danh mục các khiếm khuyết C++ tiêu biểu được viết một cách có chủ đích và mỗi lỗi được đối chiếu với bản Rust tương ứng để ghi nhận liệu nó bị từ chối tại thời điểm biên dịch, chỉ bị bắt tại thời điểm chạy, hay bị bỏ sót hoàn toàn. RQ4 được trả lời bằng cách xây dựng một nguyên mẫu FFI C++-sang-Rust hoạt động được và đo kích thước của đường ghép tương tác, phạm vi của `unsafe`, và chi phí băng qua ranh giới cho mỗi lời gọi. Toàn bộ quy trình (testbed, các khối lượng công việc, và cách xử lý thống kê) được trình bày trong Chương 3.

1.7 Đóng góp, kết quả và giới hạn

Luận văn đóng góp (i) một khung đo hiệu năng có thể tái lập, phù hợp với viễn thông cho các khối lượng công việc mặt điều khiển; (ii) một đặc tả chung đã đóng băng và hai bản hiện thực đã kiểm thử tương đương của một phân hệ mặt điều khiển RAN bằng C++ và Rust; (iii) một phép so sánh định lượng nhấn mạnh độ trễ đuôi và tính dự đoán được bên cạnh thông lượng trung bình; (iv) một phân loại các lớp lỗi thu được qua tiêm lỗi có kiểm soát; và (v) các hướng dẫn kỹ thuật và di trú dựa trên bằng chứng cho việc lựa chọn ngôn ngữ trong quá trình tiến hoá RAN gốc-đám-mây. Các phát hiện chủ chốt như sau. Về *an toàn* (RQ2), mỗi một trong các lớp lỗi C++ được tiêm vào đều bị ngăn chặn tại thời điểm biên dịch trong Rust an toàn, trong khi ở C++ mỗi lỗi đều biên dịch được và chỉ bị bắt, nếu có, tại thời điểm chạy và chỉ với đúng bộ dò lỗi; kết quả này là cuối cùng. Về *di trú* (RQ4), đường ghép FFI nhỏ, giữ cho lõi không cần `unsafe`, và không thêm chi phí đo được nào cho mỗi lời gọi — một lần băng qua ranh giới dưới một nano-giây so với một thao tác lớn hơn khoảng năm bậc độ lớn — nên chi phí áp dụng thực sự là thiết kế API, chứ không phải phụ trội lúc chạy; kết quả này cũng là cuối cùng. Về *hiệu năng* (RQ1, RQ3), các phép đo sơ bộ thu thập trên một môi trường phát triển cho thấy hai bản hiện thực bám sát nhau đến điểm bão hoà, với khác biệt chỉ giới hạn ở phần đuôi xa nơi bộ cấp phát bộ nhớ quan trọng ngang với ngôn ngữ; các con số này là sơ bộ vì các tuyên bố về độ trễ đuôi đòi hỏi testbed chạy trực tiếp trên phần cứng (bare-metal) được mô tả trong Chương 3. Các giới hạn chính là việc dùng một khối lượng công việc tổng hợp, một phân hệ đơn giản hoá duy nhất, và thiên lệch do một tác giả duy

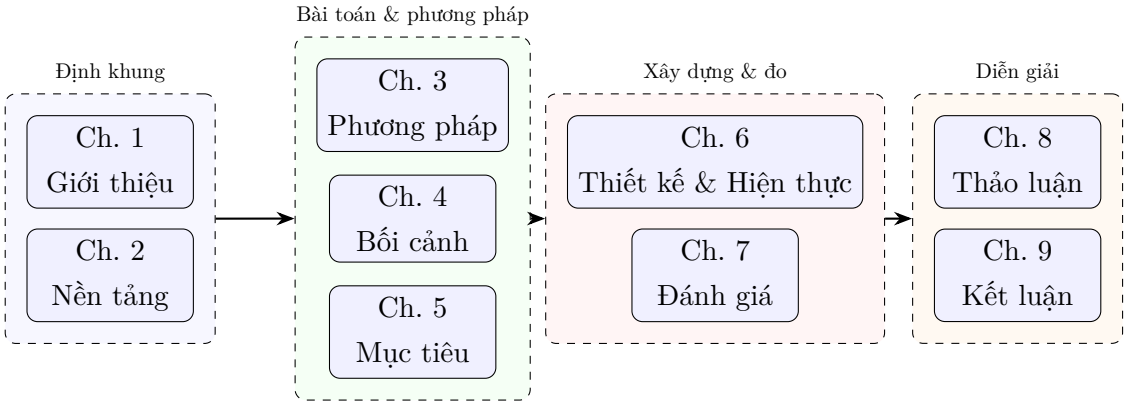
nhất hiện thực; những điều này được bàn chi tiết trong Chương 8.

1.8 Ý nghĩa và đối tượng hưởng lợi

Các kết quả được thiết kế để có thể hành động trực tiếp. Đối với các nhà cung cấp RAN và nhà khai thác mạng gốc-đám-mây, luận văn cung cấp bằng chứng có kiểm soát cho câu hỏi làm điều kiện tiên quyết cho mọi quyết định về ngôn ngữ. Câu hỏi đó không phải là "Rust có nhanh trên trung bình không?" mà là "Rust có giữ được độ trễ đuôi và tính dự đoán được khi quá tải không, và việc áp dụng tăng dần tốn kém bao nhiêu?", được trả lời cho một phân hệ tiêu biểu cho mã mặt điều khiển sản xuất. Đối với cuộc thảo luận về tiêu chuẩn và chính sách xoay quanh các lộ trình an toàn bộ nhớ [11, 25], nó đặt một mệnh lệnh tổng quát lên nền tảng của một điểm dữ liệu viễn thông cụ thể. Đối với cộng đồng nghiên cứu, đặc tả đã đóng băng, các bản hiện thực đã kiểm thử tương đương, khung đo hiệu năng có thể tái lập, và phân loại các lớp lỗi tạo thành một điểm khởi đầu tái sử dụng được cho các nghiên cứu ngôn ngữ và hệ thống về sau. Và đối với các kỹ sư đang hành nghề, danh mục lỗi và nguyên mẫu di trú đóng vai trò như những ví dụ thực chứng về những gì các bảo đảm của Rust mua được và không mua được trong lĩnh vực này.

1.9 Bố cục luận văn

Chương 2 trình bày nền tảng về phần mềm mặt điều khiển RAN, về C++, và về mô hình an toàn của Rust. Chương 3 phát biểu cách tiếp cận nghiên cứu. Chương 4 mô tả bối cảnh bài toán: phân hệ Quản lý Ngữ cảnh UE, các lớp lỗi liên quan, và tính dự đoán được của độ trễ đuôi. Chương 5 đặt ra các mục tiêu, phạm vi và ranh giới. Chương 6 trình bày thiết kế và hiện thực của cả hai bản. Chương 7 mô tả thiết lập thực nghiệm và báo cáo phần đánh giá. Chương 8 bàn về các phát hiện, công trình liên quan, và các mối đe doạ đối với tính hợp lệ. Chương 9 kết luận và phác thảo công việc tương lai. Hình 1.1 tóm tắt cấu trúc này trong một cái nhìn tổng quan.



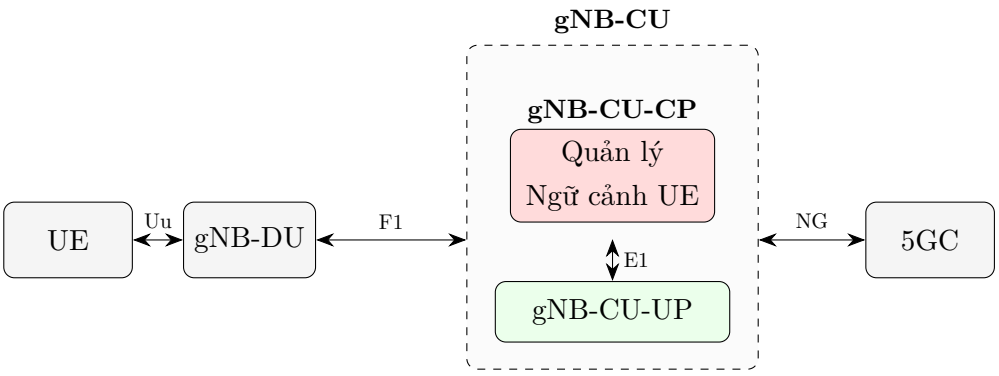
Hình 1.1: Bản đồ lộ trình của luận văn. Các chương tiến triển từ việc định khung bài toán, qua phương pháp nghiên cứu và bối cảnh bài toán, đến các sản phẩm được kiến tạo và phần đánh giá chúng, và cuối cùng đến diễn giải và kết luận.

2. Nền tảng: Các ngôn ngữ hệ thống cho phần mềm nhạy độ trễ

2.1 Các hệ thống mặt điều khiển RAN và gNB-CU-CP

Kiến trúc NG-RAN tách gNB thành một đơn vị trung tâm (gNB-CU) và các đơn vị phân tán, với mặt điều khiển (gNB-CU-CP) đảm nhiệm RRC và quản lý kết nối cho từng UE [1, 3, 4]. Các hệ thống mặt điều khiển như vậy phải thoả mãn những yêu cầu phi chức năng khắt khe: tốc độ sự kiện rất cao (di động UE, các cơn bão báo hiệu), các giới hạn độ trễ ngặt nghèo và dự đoán được mà các đặc tả dịch vụ 5G định lượng thành các KPI [5], phần mềm tồn tại lâu dài với độ phức tạp tiến hoá liên tục, và các khối lượng công việc tương tranh nặng nề. Trong các hệ thống này, hành vi trường hợp xấu nhất và phần đuôi, chứ không phải giá trị trung bình, quyết định liệu các hạn chót có được đáp ứng hay không — điều này thúc đẩy trọng tâm về tính dự đoán được của luận văn.

Về mặt chức năng, gNB được phân rã theo hai trục trực giao. Trục thứ nhất là sự tách *trung tâm/phân tán*: một đơn vị trung tâm (gNB-CU) chứa các tầng cao hơn của ngăn xếp giao thức, trong khi một hoặc nhiều đơn vị phân tán (gNB-DU) chứa các tầng thấp hơn, nhạy thời gian hơn, gần với vô tuyến. Trục thứ hai là sự tách *mặt điều khiển/mặt người dùng* bên trong đơn vị trung tâm, tách chức năng mặt điều khiển (gNB-CU-CP) khỏi chức năng mặt người dùng (gNB-CU-UP). Các phần này giao tiếp qua các giao diện chuẩn hoá — NG hướng tới lõi 5G, F1 giữa CU và DU, và E1 giữa CU-CP và CU-UP [1, 3, 2]. Các sáng kiến Open-RAN chuẩn hoá các giao diện này hơn nữa để mỗi phần có thể do một nhà cung cấp khác nhau cung cấp và chạy như phần mềm trên phần cứng phổ thông [27]. Hình 2.1 đặt phân hệ được nghiên cứu trong luận văn này — Quản lý Ngữ cảnh UE (UECM) — bên trong gNB-CU-CP: nó sở hữu trạng thái mặt điều khiển theo từng UE và làm trung gian cho các thủ tục báo hiệu RRC và di động, đó là lý do vì sao tính đúng đắn và hành vi định thời của nó tiêu biểu cho toàn bộ mặt điều khiển.

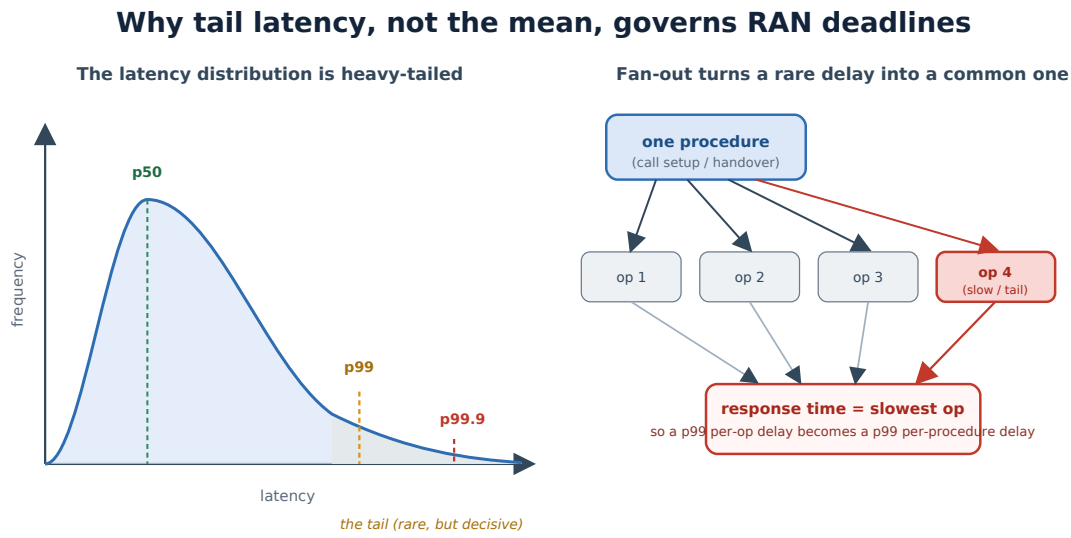


Hình 2.1: Vị trí của phân hệ được nghiên cứu trong 5G NG-RAN. gNB được tách thành một đơn vị phân tán (gNB-DU) và một đơn vị trung tâm (gNB-CU), và đơn vị trung tâm lại được tách thành các chức năng mặt điều khiển (gNB-CU-CP) và mặt người dùng (gNB-CU-UP), kết nối bằng các giao diện chuẩn hoá F1, E1 và NG. Quản lý Ngữ cảnh UE (được tô sáng) là bộ quản lý trạng thái mặt điều khiển theo từng UE bên trong gNB-CU-CP mà luận văn này hiện thực lại và đo đạc.

2.2 Độ trễ, thông lượng và tính dự đoán được trong phần mềm thời gian thực

Các hệ thống nhạy độ trễ được đánh giá ít hơn qua khối lượng công việc chúng làm trên trung bình mà nhiều hơn qua mức độ *dự đoán được* khi chúng làm việc đó. Cần phân biệt ba thước đo. *Thông lượng* là tốc độ hoàn thành thao tác được duy trì; *độ trễ* là thời gian một thao tác riêng lẻ cần; và *độ dao động* (jitter) là sự biến thiên của độ trễ đó. Vì một hệ thống có thể có độ trễ trung bình xuất sắc mà vẫn thường xuyên lỡ hạn chót, các con số có ý nghĩa là các độ trễ *đuôi* ở phân vị cao — phân vị thứ 99 (p99) và 99,9 (p99.9) — chứ không phải giá trị trung bình. Ở quy mô một mạng, độ trễ đuôi chi phối hành vi mà người dùng cảm nhận được: khi một hành động của người dùng phân nhánh (fan out) thành nhiều thao tác nội bộ, thao tác chậm nhất trong số đó quyết định thời gian đáp ứng, nên những trì hoãn hiếm gặp ở mỗi thao tác trở thành những trì hoãn phổ biến ở mỗi yêu cầu [12]. Phần mềm trung tâm dữ liệu và mạng hiện đại ngày càng vận hành trong một chế độ nơi các độ trễ liên quan được đo bằng micro-giây, một dải mà ở đó các chi phí phụ trội thông thường của hệ điều hành và phần mềm không còn không đáng kể [9]. Các nghiên cứu đo đạc chi tiết cho thấy rằng, ngay cả ở mức sử dụng vừa phải, các hiệu ứng ở mức phần cứng, hệ điều hành và ứng dụng như nhiễu nền, lập lịch, định tuyến ngắt, và các trạng thái tiết kiệm năng lượng làm phồng phần đuôi vượt xa những gì các mô hình xếp hàng dự đoán [21].

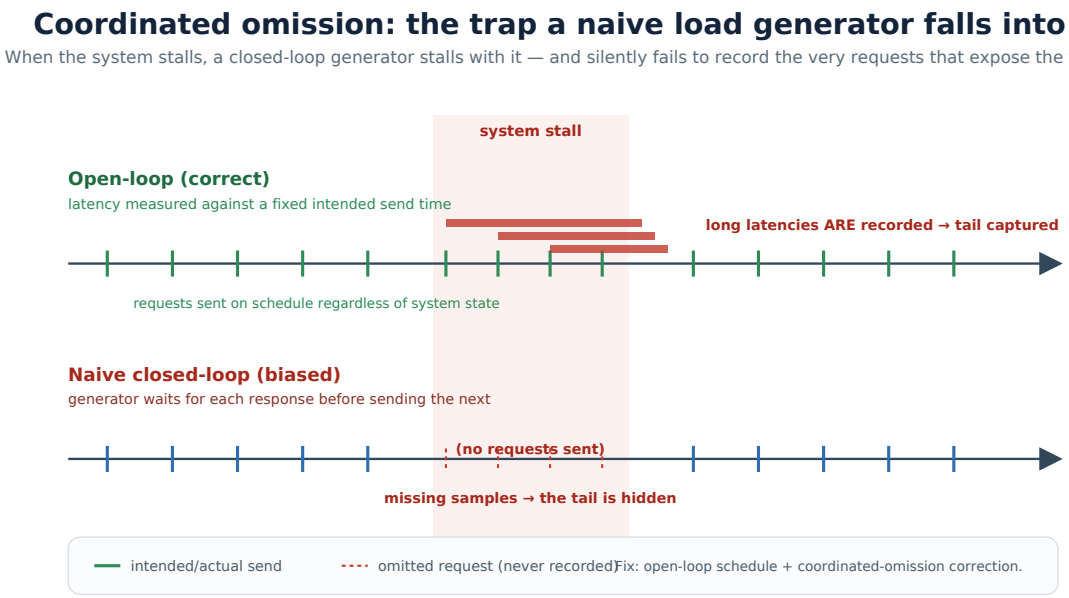
Các hệ thống thời gian thực theo quy ước được phân hạng theo hậu quả của một hạn chót bị lỡ: trong thời gian thực *cứng* (hard) một lần lỡ là một thất bại, trong thời gian thực *chắc* (firm) một kết quả trễ là vô dụng nhưng không thảm khốc,



Hình 2.2: Vì sao phần đuôi, chứ không phải giá trị trung bình, chi phối các hạn chót của RAN. Phân bố độ trễ có đuôi nặng (trái), nên các phân vị p99/p99.9 nằm xa trung vị. Vì một thủ tục đơn lẻ phân nhánh thành nhiều thao tác phụ thuộc mà thành viên chậm nhất quyết định thời điểm hoàn thành (phải), một trì hoãn hiếm gặp ở mỗi thao tác trở thành phổ biến ở mỗi thủ tục; ở quy mô toàn đội, một sự kiện đuôi một-phần-nghìn là chuyện thường xuyên xảy ra ở đâu đó tại mọi thời điểm.

và trong thời gian thực *mềm* (soft) một lần lỡ chỉ làm giảm chất lượng. Một mặt điều khiển RAN được nhìn nhận tốt nhất như thời gian thực mềm-đến-chắc: một đáp ứng báo hiệu trễ thi thoảng làm giảm dịch vụ chứ không gây nguy hiểm cho nó, nhưng ở quy mô toàn đội và dưới các cơn bão báo hiệu, những sự suy giảm như vậy tích tụ thành một vấn đề độ tin cậy trên phạm vi toàn mạng. Điều này bị làm trầm trọng thêm bởi hành vi xếp hàng: khi tải đưa vào tiến gần đến năng lực phục vụ, trễ xếp hàng tăng mạnh và đuôi độ trễ bùng nổ gần điểm bão hoà. Do đó, việc đặc trưng hoá một hệ thống đòi hỏi điều khiển nó không chỉ ở tải danh định mà *vào trong vùng quá tải*, đó là lý do vì sao phần đánh giá trong luận văn này quét tải lên tới và vượt qua điểm bão hoà.

Bản thân việc đo độ trễ đuôi cho đúng cũng là một cạm bẫy. Một bộ sinh tải vòng-kín (closed-loop) ngây thơ, chờ mỗi đáp ứng trước khi phát yêu cầu tiếp theo, mắc phải *sự bỏ sót phối hợp* (coordinated omission): khi hệ thống khựng lại, bộ sinh tải khựng theo và đơn giản là không ghi nhận nhiều yêu cầu lẽ ra đã đến trong lúc khựng, qua đó che giấu một cách có hệ thống chính cái đuôi mà nó đáng lẽ phải phơi bày. Do đó, phép đo đúng đắn đòi hỏi một mô hình vòng-hở (open-loop) với một lịch dự định, hiệu chỉnh cho sự bỏ sót phối hợp, các biểu đồ độ trễ dải-động-cao giữ được độ phân giải qua nhiều bậc độ lớn, và báo cáo chặt chẽ về mặt thống kê với các lần chạy lặp lại và các khoảng tin cậy thay vì các ước lượng điểm đơn lẻ [16]. Các



Hình 2.3: Sự bỏ sót phối hợp. Một bộ sinh tải vòng-hở (trên) gửi theo một lịch dự định cố định bất kể trạng thái hệ thống, nên các yêu cầu phát ra trong lúc khựng vẫn được ghi nhận với độ trễ đầy đủ và phần đuôi được nắm bắt. Một bộ sinh tải vòng-kín ngây thơ (dưới) chờ mỗi đáp ứng trước khi gửi tiếp, nên khi hệ thống khựng nó cũng khựng theo và không bao giờ phát các yêu cầu bị trì hoãn — âm thầm bỏ sót đúng những mẫu lẽ ra sẽ phơi bày phần đuôi.

kỹ thuật này được áp dụng trong Chương 7.

2.3 C++ cho các hệ thống nhạy độ trễ

C++ là ngôn ngữ mặc định cho phần mềm mặt điều khiển RAN vì những lý do chính đáng. Nó cung cấp các trừu tượng không-chi-phí-phụ-trội (zero-overhead), khả năng kiểm soát trực tiếp bố cục và cấp phát bộ nhớ, sự huỷ đối tượng tất định thông qua thành ngữ RAII (Resource Acquisition Is Initialization), và các trình biên dịch, thư viện, công cụ định hình (profiling) trưởng thành, tất cả mà không có bộ thu gom rác có thể chèn vào những khoảng dừng khó lường. C++ hiện đại (C++11 trở về sau) bổ sung các con trỏ thông minh (`std::unique_ptr`, `std::shared_ptr`) tự động hoá quyền sở hữu, ngữ nghĩa di chuyển (move semantics) tránh các bản sao không cần thiết, và một mô hình bộ nhớ chính thức với `std::atomic` cho tương tranh dựa trên khoá và không khoá (lock-free). Các tính năng này giúp có thể viết mã hệ thống nhanh và, với kỷ luật, an toàn ở mức hợp lý.

Khó khăn là ở chỗ tính an toàn trong C++ vẫn là trách nhiệm của lập trình viên chứ không phải một thuộc tính mà trình biên dịch cưỡng chế. RAII và con trỏ thông minh giảm nhưng không loại bỏ các lỗi thời gian: các con trỏ và tham chiếu thô có thể sống lâu hơn các đối tượng chúng trỏ tới, các iterator có thể bị vô hiệu

hoá bởi việc biến đổi bộ chứa (container), các đối tượng có thể bị dùng sau khi đã di chuyển, và không gì trong hệ thống kiểu ngăn được một đối tượng khả biến bị tham chiếu bí danh (alias) và biến đổi đồng thời từ nhiều luồng. Kết quả là một danh mục khiếm khuyết đã được biết rõ — sử-dụng-sau-khi-giải-phóng, giải-phóng-hai-lần, tràn bộ đệm, tranh chấp dữ liệu, deadlock, và vô hiệu hoá iterator — dễ tạo ra và tốn kém để tìm ở quy mô viễn thông. Các lỗi hỏng bộ nhớ loại này đã là một vấn đề bảo mật dai dẳng suốt nhiều thập kỷ, và các biện pháp giảm thiểu đã triển khai vẫn chỉ là những phòng vệ một phần [30]. Mức độ phổ biến của chúng được ghi nhận rõ trong công nghiệp: các vấn đề an toàn bộ nhớ chiếm khoảng 70 % các lỗ hổng bảo mật được báo cáo tại Microsoft [22], và Google báo cáo một sự sụt giảm mạnh các lỗ hổng như vậy khi các ngôn ngữ an toàn bộ nhớ được áp dụng trong Android [32]. Danh mục cụ thể này được phát triển chi tiết trong Chương 4 và tạo cơ sở cho nghiên cứu tiêm lỗi trong Chương 7.

Thực tiễn C++ giảm thiểu các khiếm khuyết này bằng công cụ chứ không phải bằng các bảo đảm của ngôn ngữ, và công cụ về bản chất là chỉ một phần. Các bộ dò lỗi động (dynamic sanitizer) như AddressSanitizer (ASan), ThreadSanitizer (TSan), và UndefinedBehaviorSanitizer phát hiện nhiều lỗi, nhưng chỉ trên các đường mã mà một kiểm thử nhất định thực sự thực thi với đúng đầu vào, và chúng áp đặt các chi phí phụ trội về thời gian chạy và bộ nhớ khiến chúng không phù hợp cho môi trường sản xuất; riêng TSan chỉ báo cáo những tranh chấp dữ liệu tình cờ xảy ra trong một lần chạy, nên một tranh chấp chỉ biểu hiện dưới một trình tự đan xen hiếm gặp có thể lọt qua không bị chú ý. Các trình phân tích tĩnh giúp ích nhưng chịu các âm tính giả (false negative) và không thể lập luận về toàn bộ chương trình. Do đó việc phát hiện là nỗ-lực-tối-đa và sau-khi-sự-việc-đã-rồi, chính là tình huống mà một ngôn ngữ với các bảo đảm tại thời điểm biên dịch muốn thay đổi.

2.4 Mô hình quyền sở hữu và an toàn của Rust

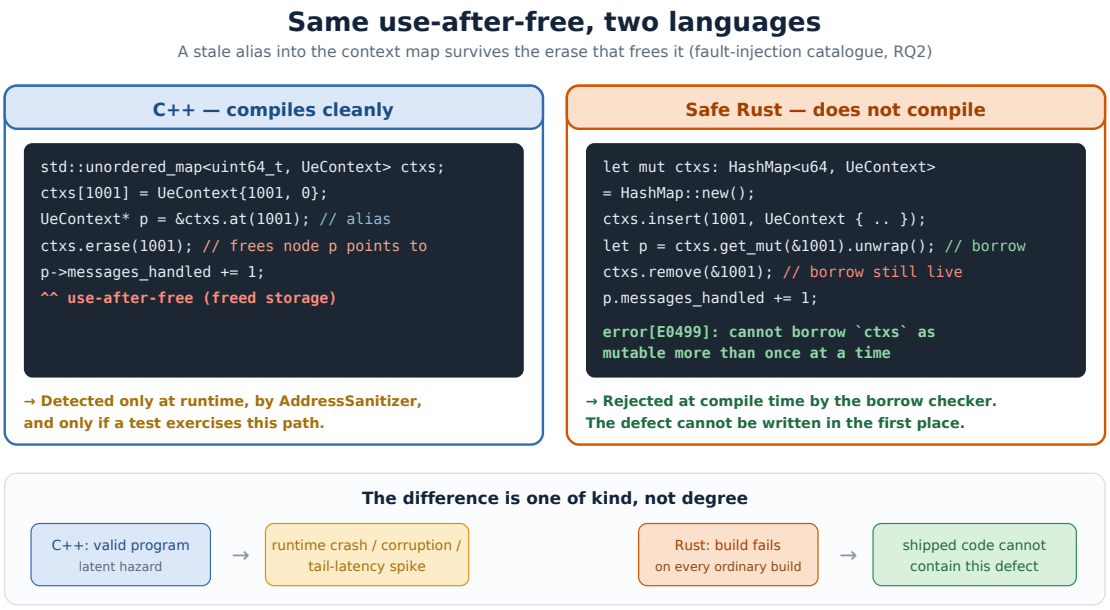
Rust cung cấp an toàn bộ nhớ và không có tranh chấp dữ liệu được cưỡng chế tại thời điểm biên dịch thông qua mô hình quyền sở hữu và mượn của nó [20], cùng với một câu chuyện tương tranh mạnh mẽ và công cụ hiện đại (Cargo, kiểm thử tích hợp). Nó là ngôn ngữ được công nghiệp hỗ trợ đầu tiên vượt qua được sự đánh đổi truyền thống giữa tính an toàn của các ngôn ngữ cấp cao và khả năng kiểm soát mức thấp của C và C++, đạt được điều này với một hệ thống kiểu dựa trên quyền sở hữu và mượn, cấm một cách tĩnh việc biến đổi trạng thái chia sẻ, đồng thời giới hạn các ngoại lệ không thể tránh khỏi vào mã `unsafe` được đóng gói sau các API an toàn [19]. Tính đúng đắn (soundness) của phân đoạn an toàn của ngôn ngữ, bao gồm vai trò của `unsafe` như một cửa thoát được giới hạn tường minh, đã được thiết lập

một cách hình thức bởi dự án RustBelt [18], và kỷ luật bí danh (aliasing) mà ngay cả mã `unsafe` cũng phải tôn trọng đã được cho một ngữ nghĩa vận hành chính xác [17]. Bằng chứng thực nghiệm ủng hộ mô hình này trong thực tế: `unsafe` được dùng một cách tiết chế và cục bộ trong các mã nguồn thực [7, 28], và một nghiên cứu về tất cả các CVE an toàn bộ nhớ của Rust phát hiện rằng mỗi CVE trong số đó đều liên quan đến mã `unsafe`, xác nhận rằng Rust an toàn giữ đúng lời hứa của nó [33]. Điều lưu ý là tính không an toàn lan truyền qua các phụ thuộc: mặc dù ít hơn một phần ba số thư viện dùng `unsafe` trực tiếp, một tỉ lệ lớn hơn nhiều không thể được kiểm tra tĩnh đầy đủ vì `unsafe` ẩn ở đâu đó trong chuỗi lời gọi của chúng [14], đây là một lập luận nữa cho việc giữ lõi nhạy-an-toàn của một phân hệ không có `unsafe`, như luận văn này làm. Điều then chốt đối với một bối cảnh nhạy độ trễ là tính an toàn này được cưỡng chế bởi hệ thống kiểu và trình kiểm tra mượn (borrow checker) chứ không phải bởi công cụ đo lường thời gian chạy, và được dự định là *không-chi-phí* (zero-cost): nghiên cứu hệ thống lập luận rằng Rust đạt được an toàn mà không có chi phí phụ trội của thu gom rác hay kiểm tra thời gian chạy, và thậm chí cho phép các kỹ thuật hiệu quả bất khả thi trong C [8]. Liệu tuyên bố không-chi-phí đó có đúng cho một khối lượng công việc mặt điều khiển viễn thông hay không chính là thuộc tính mà luận văn này đánh giá.

Cụ thể, trình kiểm tra mượn từ chối tại thời điểm biên dịch chính cái lỗi sử-dụng-sau-khi-giải-phóng đã phác trong phần giới thiệu. Chương trình sau không dựng được, vì `ctx` bị di chuyển (bị drop) trong khi một tham chiếu tới nó vẫn còn sống:

```
let ctx = UeContext::new(42);
let r = &ctx;       // immutable borrow of ctx
drop(ctx);          // move/free ctx ...
use_ctx(r);         // error[E0505]: cannot move out of `ctx`
                    //               because it is borrowed
```

Cùng kỷ luật đó mở rộng sang tương tranh. Rust mã hoá an toàn luồng trong hệ thống kiểu thông qua các trait đánh dấu `Send` và `Sync`: một giá trị chỉ có thể băng qua ranh giới luồng nếu kiểu của nó là `Send`, và chỉ có thể được chia sẻ bằng tham chiếu qua các luồng nếu nó là `Sync`. Do đó, việc chia sẻ một giá trị cho phép biến đổi không đồng bộ hoá là một lỗi biên dịch, chứ không phải một tranh chấp tiềm ẩn — thuộc tính mà Rust quảng bá là "tương tranh không sợ hãi" (fearless concurrency). Việc xử lý lỗi cũng tường minh không kém: các thao tác có thể thất bại trả về một giá trị `Result` mà bên gọi phải xử lý, thay vì nâng một ngoại lệ tháo lui (unwind) qua dòng điều khiển ẩn, điều này giữ cho chi phí và dòng điều khiển của đường thất bại luôn hiển thị — một lợi thế trong mã nhạy độ trễ nơi những bất ngờ ở phần đuôi

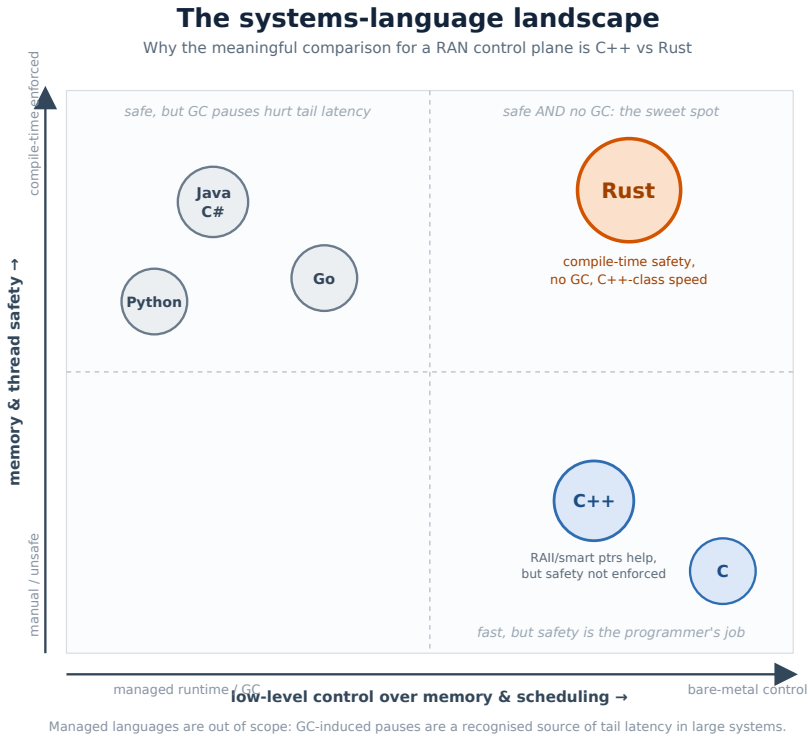


Hình 2.4: Cùng một lỗi sử-dụng-sau-khi-giải-phóng trong C++ và trong Rust an toàn. Bí danh cũ trỏ vào bản đồ ngữ cảnh sống sót qua thao tác `erase/remove` giải phóng nó: trong C++ điều này biên dịch được và chỉ bị bắt tại thời điểm chạy (và chỉ dưới AddressSanitizer, trên một đường mà một kiểm thử thực thi), trong khi trong Rust an toàn trình kiểm tra mượn từ chối nó tại thời điểm biên dịch (`E0499`), nên lỗi này hoàn toàn không thể viết ra được. Sự khác biệt là về bản chất, không phải về mức độ.

mới là thứ quan trọng.

2.5 Bức tranh toàn cảnh các ngôn ngữ hệ thống

Rust không phải là ngôn ngữ an toàn bộ nhớ duy nhất, nhưng nó chiếm một vị trí đặc biệt cho bài toán này. Các ngôn ngữ có runtime quản lý (managed) như Java, C# và Go đạt được an toàn bộ nhớ thông qua một runtime và thu gom rác. Đó là một đánh đổi tuyệt vời cho nhiều loại phần mềm, nhưng lại là một lựa chọn tồi cho một mặt điều khiển nhạy-độ-trễ-đuôi: thu gom rác gây ra các khoảng dừng và biến thiên định thời vốn là một nguồn gây độ trễ đuôi đã được thừa nhận trong các hệ thống lớn [12], và một runtime quản lý cũng nhường lại quyền kiểm soát mức thấp đối với bố cục bộ nhớ và lập lịch mà lĩnh vực này dựa vào. Ở thái cực khác, C cũng không an toàn bộ nhớ như C++ trong khi cung cấp ít trừu tượng hơn để xây dựng các hệ thống lớn. Rust là ngôn ngữ được hỗ trợ rộng rãi đầu tiên kết hợp an toàn bộ nhớ và luồng, được cưỡng chế tại thời điểm biên dịch, với sự vắng mặt của bộ thu gom rác và hiệu năng nhắm tới việc cạnh tranh được với C++ [19, 8]; các khảo sát đo hiệu năng đa ngôn ngữ đặt thông lượng và mức dùng bộ nhớ của Rust ngang với

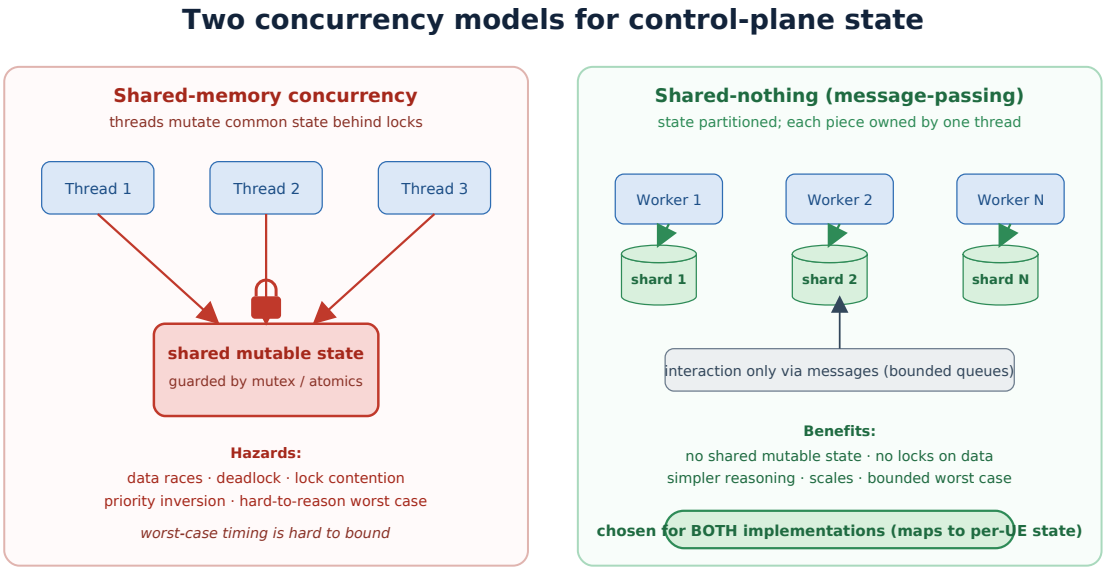


Hình 2.5: Bức tranh toàn cảnh các ngôn ngữ hệ thống theo hai trục: khả năng kiểm soát mức thấp đối với bộ nhớ và lập lịch (ngang) và an toàn bộ nhớ/luồng (dọc). Các ngôn ngữ có runtime quản lý (Java, C#, Go, Python) an toàn nhưng trả giá bằng chi phí thu gom rác làm hại độ trễ đuôi; C và C++ cho khả năng kiểm soát sát phần cứng nhưng để tính an toàn cho lập trình viên. Rust chiếm điểm ngọt — an toàn tại thời điểm biên dịch mà không có bộ thu gom rác — đó là lý do vì sao phép so sánh có ý nghĩa cho một mặt điều khiển RAN là C++ đối đầu Rust.

C và C++ trong khi nó vẫn giữ các bảo đảm an toàn của mình [10]. Chính sự kết hợp đó là lý do vì sao phép so sánh có ý nghĩa cho phần mềm mặt điều khiển RAN là C++ đối đầu Rust, và vì sao các ngôn ngữ có runtime quản lý nằm ngoài phạm vi ở đây.

2.6 Các mô hình tương tranh cho phần mềm mặt điều khiển

Cách cấu trúc tương tranh định hình cả tính đúng đắn lẫn độ trễ đuôi. Mô hình chủ đạo là tương tranh *bộ-nhớ-chia-sẻ* (shared-memory), trong đó các luồng biến đổi trạng thái chung được bảo vệ bằng mutex và atomic; nó linh hoạt nhưng là nguồn gốc của các tranh chấp, deadlock, tranh chấp khoá (lock contention) và đảo ngược ưu tiên (priority inversion), và định thời trường hợp xấu nhất của nó khó lập luận. Phương án thay thế là tương tranh *truyền-thông-điệp* (message-passing) hay *không-chia-sẻ-gì* (shared-nothing), trong đó trạng thái được phân vùng sao cho mỗi phần được sở hữu



Hình 2.6: Hai mô hình tương tranh cho trạng thái mặt điều khiển. Tương tranh bộ-nhớ-chia-sẻ (trái) để các luồng biến đổi trạng thái chung sau các khoá và là nguồn gốc của tranh chấp dữ liệu, deadlock và tranh chấp khoá, với một trường hợp xấu nhất khó giới hạn. Truyền-thông-điệp không-chia-sẻ-gì (phải) phân vùng trạng thái sao cho mỗi mảnh được sở hữu bởi một worker duy nhất và mọi tương tác diễn ra qua các hàng đợi có chặn; điều này loại bỏ các khoá trên dữ liệu ngữ cảnh và ánh xạ tự nhiên lên việc phân vùng theo từng UE, và là mô hình được áp dụng cho cả hai bản hiện thực.

bởi một luồng duy nhất và mọi tương tác diễn ra qua các thông điệp, như trong mô hình actor và trong các thiết kế phân mảnh (sharded). Không-chia-sẻ-gì đánh đổi một phần tính linh hoạt để lấy khả năng mở rộng và sự lập luận đơn giản hơn nhiều, và nó ánh xạ tự nhiên lên việc phân vùng theo từng UE trong một mặt điều khiển, nơi trạng thái của mỗi UE phần lớn là độc lập.

Một lựa chọn thứ hai, trực giao, là cách công việc được xếp hàng giữa bên sản xuất và bên tiêu thụ. Các hàng đợi không chặn (unbounded) che giấu quá tải và để độ trễ cùng bộ nhớ tăng không giới hạn; các hàng đợi *có chặn* (bounded) với hoặc chặn nghẽn (backpressure) hoặc loại bỏ bớt (shedding) giới hạn bộ nhớ trường hợp xấu nhất và làm cho quá tải trở nên hữu hình, điều thiết yếu cho độ trễ đuôi dự đoán được. Các cấu trúc không khoá (lock-free) có thể giảm tranh chấp nhưng nổi tiếng là khó hiện thực cho đúng. Những cân nhắc này trực tiếp thúc đẩy kiến trúc được áp dụng cho cả hai bản hiện thực trong Chương 6: một thiết kế không-chia-sẻ-gì, một-luồng-trên-mỗi-mảnh (thread-per-shard) với các hàng đợi nhiều-bên-sản-xuất/một-bên-tiêu-thụ có chặn và chặn nghẽn bằng cách chặn, được chọn sao cho bất kỳ khác biệt độ trễ đo được nào cũng phản ánh ngôn ngữ chứ không phải một chiến lược tương tranh khác nhau.

2.7 Khả năng tương tác giữa C++ và Rust

Việc áp dụng một ngôn ngữ mới trong một hệ thống lớn sẵn có nhất thiết phải là tăng dần: một thành phần Rust phải cùng tồn tại với, và có thể được gọi từ, một mã nguồn C++ đáng kể. Mẫu số chung cho khả năng tương tác như vậy là giao diện nhị phân ứng dụng C (C ABI). Rust có thể phơi bày các hàm với liên kết `extern "C"` và trao ra các *handle mờ* (opaque handle) tới các đối tượng do Rust sở hữu, mà C++ sau đó gọi qua một header C; công cụ tự động hoá một phần việc này, bao gồm `bindgen` (sinh ràng buộc Rust từ header C), `cbindgen` (sinh header C từ Rust), và crate `cxx` (một cầu nối hai chiều an toàn hơn). Khả năng tương tác không miễn phí: băng qua ranh giới là bước ra ngoài ngôn ngữ được kiểm tra, nên đường ghép (seam) phải chứa mã `unsafe`; truyền bất cứ thứ gì ngoài các giá trị đơn giản đòi hỏi việc sắp xếp (marshalling); và hình dạng của giao diện ngoại lai phải được thiết kế một cách có chủ đích. Kích thước của đường ghép đó, nó tập trung bao nhiêu `unsafe`, và chi phí băng qua ranh giới cho mỗi lời gọi chính là các đại lượng được đo cho RQ4 trong Chương 7.

2.8 Khoảng trống nghiên cứu

Các mục trước đã thiết lập ba điều. Thứ nhất, mặt điều khiển RAN là một lĩnh vực thời gian thực mềm-đến-chắc, tương tranh cao, nhạy độ trễ, trong đó hành vi phần đuôi khi quá tải, chứ không phải thông lượng trung bình, chi phối chất lượng mà mạng cảm nhận. Thứ hai, C++ cung cấp khả năng kiểm soát và hiệu năng mà lĩnh vực này đòi hỏi nhưng để cả những lớp lỗi bộ nhớ và tương tranh cho việc phát hiện nỗ-lực-tối-đa, sau-khi-sự-việc-đã-rồi. Thứ ba, Rust hứa hẹn ngăn chặn các lớp đó tại thời điểm biên dịch mà không có chi phí thời gian chạy, với một con đường áp dụng tăng dần đáng tin cậy qua khả năng tương tác C-ABI. Thứ còn thiếu trong tài liệu là bằng chứng *trực tiếp, có kiểm soát* cho một phân hệ mặt điều khiển tiêu biểu: liệu Rust có thực sự giữ được độ trễ đuôi và tính dự đoán được khi quá tải trước C++ hiện đại hay không, chính xác những lớp lỗi nào nó loại bỏ và những lớp còn lại bị bắt ở giai đoạn nào, mô hình an toàn của nó tốn kém bao nhiêu về thông lượng và tài nguyên, và việc áp dụng tăng dần tốn kém bao nhiêu trong thực tế. Lấp khoảng trống này chính là mục đích của bốn câu hỏi nghiên cứu, được theo đuổi với phương pháp trình bày trong Chương 3.

3. Phương pháp nghiên cứu

3.1 Cách tiếp cận nghiên cứu

Luận văn tuân theo một cách tiếp cận nghiên cứu thực nghiệm, có tính thí nghiệm, với một yếu tố kiến tạo (khoa học thiết kế). Một sản phẩm phần mềm — hai bản hiện thực tương đương về chức năng của phân hệ cùng với một khung đo hiệu năng — được xây dựng dựa trên một đặc tả đã đóng băng duy nhất rồi được đánh giá dưới các thí nghiệm có kiểm soát. Bằng chứng được tam giác hoá (triangulate) từ ba phương pháp bổ trợ: đo hiệu năng định lượng dưới một tải được quét (giải quyết RQ1 và RQ3), một nghiên cứu tiêm lỗi có kiểm soát về việc phát hiện lỗi (RQ2), và một nghiên cứu tình huống di trú FFI vừa định tính vừa định lượng (RQ4). Tính công bằng được xem là mối quan tâm hàng đầu: vì mục tiêu là cô lập các ảnh hưởng của ngôn ngữ, nghiên cứu cố định đặc tả, các thuật toán, mô hình tương tranh, bộ cấp phát bộ nhớ và cấu hình biên dịch trên cả hai bản hiện thực, và báo cáo độ biến thiên qua nhiều lần chạy lặp lại.

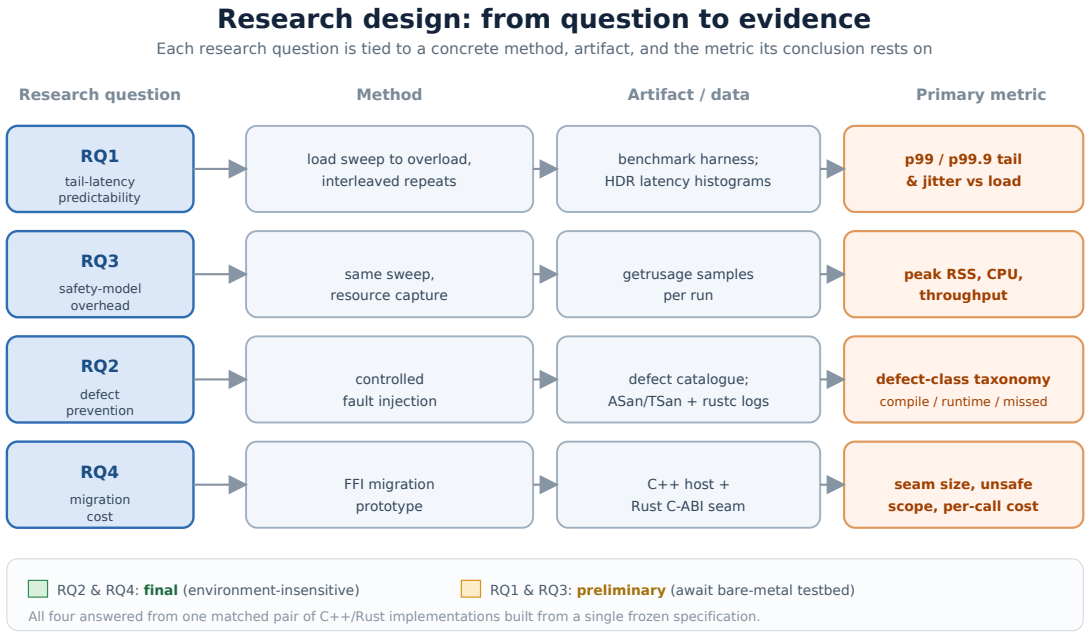
Bảng 3.1 ánh xạ mỗi câu hỏi nghiên cứu tới phương pháp, sản phẩm, dữ liệu và thước đo được dùng để trả lời nó, làm cho chuỗi từ câu hỏi đến bằng chứng trở nên tường minh và có thể truy vết.

3.2 Một thí nghiệm kiến tạo, so sánh

Bốn câu hỏi nghiên cứu mang tính nhân quả và so sánh: mỗi câu hỏi hỏi một kết quả (độ trễ đuôi, các lỗi được ngăn chặn, mức dùng tài nguyên, hay chi phí di trú) phụ thuộc thế nào vào một biến độc lập duy nhất — ngôn ngữ hiện thực. Trả lời những câu hỏi như vậy một cách đáng tin đòi hỏi ngôn ngữ phải là khác biệt hệ thống *duy nhất* giữa hai đối tượng được kiểm tra. Điều này loại trừ hình thức so sánh ngôn ngữ phổ biến nhất trong tài liệu, trong đó hai chương trình được viết độc lập, hay hai dự án không liên quan, được đo và các khác biệt của chúng được quy cho ngôn ngữ; những so sánh như vậy trộn lẫn ngôn ngữ với thiết kế, các thư viện, cùng ý định và kỹ năng của các tác giả. Do đó, luận văn áp dụng một chiến lược kiến tạo (khoa học

Bảng 3.1: Thiết kế nghiên cứu: mỗi câu hỏi nghiên cứu được trả lời như thế nào. Mỗi câu hỏi gắn với một phương pháp cụ thể, một sản phẩm được công bố, dữ liệu nó tạo ra, và thước đo mà kết luận dựa vào.

RQ	Phương pháp	Sản phẩm / dữ liệu	Thước đo chính
RQ1	Quét tải đến quá tải, lặp lại đan xen	Bộ khung đo; các biểu đồ độ trễ	độ trễ đuôi p99/p99.9 và độ dao động theo tải đưa vào
RQ3	Cùng phép quét, thu tài nguyên	Các mẫu `getrusage` mỗi lần chạy	RSS đỉnh, mức dùng CPU, thông lượng
RQ2	Tiêm lỗi có kiểm soát	Danh mục lỗi; nhật ký ASan/TSan và `rustc`	phân loại lớp lỗi (biên dịch / thời gian chạy / không phát hiện)
RQ4	Nguyên mẫu di trú FFI	Host C++ + đường ghép C-ABI Rust	kích thước đường ghép, phạm vi `unsafe`, chi phí tương tác mỗi lời gọi



Hình 3.1: Thiết kế nghiên cứu dưới dạng một luồng, cho thấy với mỗi câu hỏi nghiên cứu là phương pháp, sản phẩm và dữ liệu nó tạo ra, và thước đo chính mà kết luận của nó dựa vào. Cả bốn câu hỏi đều được trả lời từ một cặp bản hiện thực C++/Rust khớp nhau, được xây dựng từ một đặc tả đã đóng băng duy nhất; RQ2 và RQ4 không nhạy với môi trường và được báo cáo là kết quả cuối cùng, trong khi RQ1 và RQ3 là sơ bộ cho tới chiến dịch testbed chạy trực tiếp trên phần cứng.

thiết kế): thay vì định vị hai hệ thống có sẵn, nó *xây dựng* đối tượng nghiên cứu như một cặp bản hiện thực khớp nhau được dẫn xuất từ một đặc tả đã đóng băng, sao cho phép so sánh là giữa hai cách mã hoá của cùng một thiết kế chứ không phải giữa hai thiết kế. Sản phẩm — đặc tả, hai bản hiện thực, và bộ khung đo — do đó đồng thời là đóng góp của luận văn và là công cụ để đo lường nó.

Thí nghiệm mang tính thực nghiệm và định lượng ở những chỗ câu hỏi cho phép có con số (RQ1, RQ3, và chi phí mỗi lời gọi của RQ4) và có hệ thống-nhưng-định-tính ở những chỗ không cho phép (sự phân biệt loại-bảo-đảm trong RQ2 và công sức thiết kế API trong RQ4). Bằng chứng được tam giác hoá một cách có chủ đích qua ba phương pháp bổ trợ chứ không dựa vào một chiến dịch duy nhất, sao cho một điểm yếu trong bất kỳ phương pháp nào — ví dụ độ nhạy với môi trường của phép đo độ trễ đuôi — không làm suy yếu các kết luận nói chung.

3.3 Các phương pháp cho bốn câu hỏi nghiên cứu

Bảng 3.1 ghép mỗi câu hỏi với một phương pháp; mục này phát biểu mỗi phương pháp làm gì và vì sao nó phù hợp.

3.3.1 Đo hiệu năng dưới một phép quét tải (RQ1, RQ3)

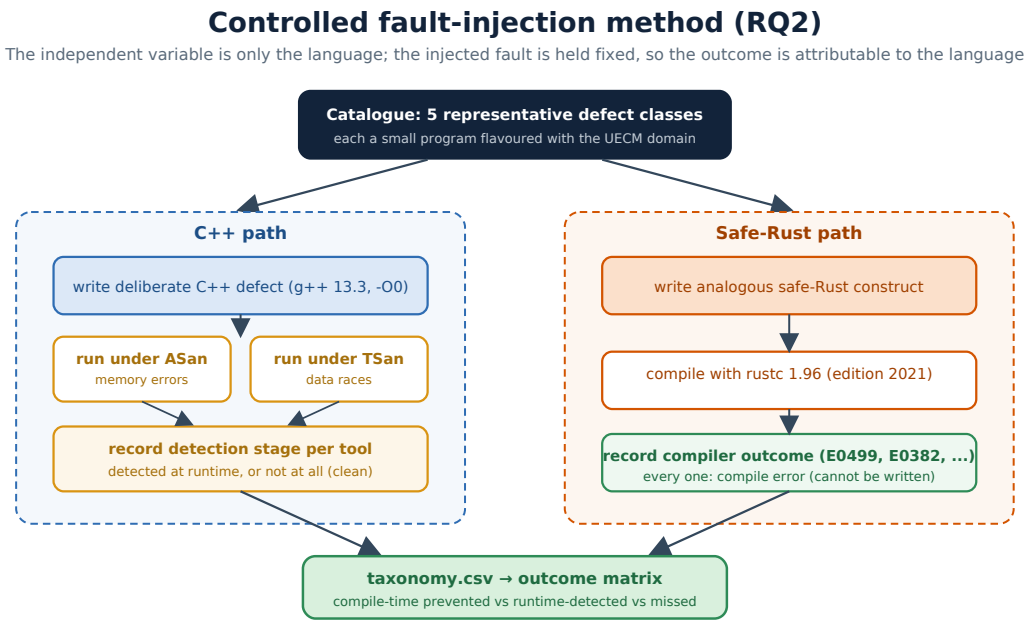
RQ1 và RQ3 được trả lời bằng cách điều khiển cả hai bản hiện thực với một khối lượng công việc tổng hợp tất định qua một phép quét tải đưa vào, từ tải nhẹ qua bão hoà và vào trong quá tải, và ghi nhận độ trễ cùng mức dùng tài nguyên ở mỗi mức. Hai quyết định thiết kế làm cho phép đo trở nên đúng đắn. Thứ nhất, bộ sinh tải tuân theo một lịch *vòng-hở* (open-loop) với một thời điểm gửi dự định cố định cho mỗi yêu cầu, và độ trễ được đo so với thời điểm dự định đó chứ không phải so với thời điểm yêu cầu thực sự được nhận vào; đây chính là cái hiệu chỉnh cho sự bỏ sót phối hợp, thiên lệch mà qua đó một bộ sinh tải vòng-kín ngây thơ khựng lại cùng hệ thống và âm thầm không ghi nhận đúng những yêu cầu lẽ ra sẽ phơi bày phần đuôi (Chương 2). Thứ hai, phép quét được đẩy vượt qua bão hoà một cách có chủ đích: hành vi mang tính quyết định về mặt vận hành của một mặt điều khiển không phải là độ trễ của nó ở tải danh định mà là sự khởi phát và hình dạng của vách quá tải (overload cliff), và điều đó chỉ hữu hình một khi tải đưa vào vượt năng lực phục vụ. Độ trễ được ghi nhận trong các biểu đồ dải-động-cao được cấu hình giống hệt nhau ở cả hai bên, và mức dùng tài nguyên (kích thước tập thường trú đỉnh, mức dùng CPU, thông lượng) được lấy mẫu qua hệ điều hành, sao cho cùng một bộ máy đo được áp dụng cho cả hai ngôn ngữ.

3.3.2 Tiêm lỗi có kiểm soát (RQ2)

RQ2 hỏi một ngôn ngữ ngăn chặn những lớp lỗi nào, và nó không thể được trả lời bằng cách đo mã không-có-lỗi: sự vắng mặt của một lỗi trong một kiểm thử đạt không nói gì về việc liệu ngôn ngữ có bắt được nó hay không. Do đó phương pháp mang tính đối kháng, tuân theo thực tiễn đã được thiết lập của tiêm lỗi phần mềm, trong đó các lỗi tiêu biểu được đưa vào một cách có chủ đích để đánh giá một hệ thống hành xử thế nào khi có mặt chúng [24]. Một danh mục các khiếm khuyết C++ tiêu biểu, rút ra từ các lớp gây hại cho phân hệ này (Chương 4), được viết một cách *có chủ đích*, mỗi lỗi là một chương trình nhỏ, tự chứa, mang màu sắc lĩnh vực (một con trỏ cũ trỏ vào bản đồ ngữ cảnh, một lần release kép, một biến đổi trong khi lặp, một cập nhật chia sẻ không đồng bộ hoá). Mỗi khiếm khuyết C++ sau đó được dựng và chạy dưới AddressSanitizer và ThreadSanitizer, ghi nhận công cụ nào, nếu có, phát hiện nó và ở giai đoạn nào; cấu trúc tương tự được viết trong Rust an toàn và biên dịch, ghi nhận kết quả của trình biên dịch. Vì biến độc lập lại chỉ là ngôn ngữ trong khi lỗi được tiêm vào được giữ cố định, khác biệt về kết quả — bị từ chối tại thời điểm biên dịch, bị phát hiện tại thời điểm chạy, hay bị bỏ sót âm thầm — quy được cho các bảo đảm của ngôn ngữ chứ không phải cho lỗi cụ thể. Kết quả là một phân loại chứ không phải một con số hiệu năng: phương pháp thiết lập *loại* bảo vệ mà mỗi ngôn ngữ cung cấp.

3.3.3 Nghiên cứu tình huống di trú qua giao diện hàm ngoại (RQ4)

RQ4 liên quan đến chi phí thực tiễn của việc áp dụng Rust bên trong một hệ thống C++ sẵn có, đây là một câu hỏi kỹ nghệ nhiều ngang với câu hỏi hiệu năng. Nó được trả lời bằng cách xây dựng một nguyên mẫu hoạt động được theo hướng di trú thực tế — một host C++ sẵn có gọi một thành phần Rust qua một giao diện nhị phân ứng dụng C — rồi đo ba đại lượng trên đó: *kích thước* của đường ghép tương tác (các điểm vào, số dòng mã, các phụ thuộc), *phạm vi* của `unsafe` mà nó buộc phải có (chỗ nào rời khỏi ngôn ngữ được kiểm tra, và liệu lõi nhạy-an-toàn có bị ảnh hưởng hay không), và *chi phí mỗi lời gọi* của việc băng qua ranh giới so với chi phí của thao tác đang được gọi. Hai đại lượng đầu mang tính cấu trúc và được đọc ra từ mã; đại lượng thứ ba được đo với cùng bộ máy biểu đồ như phép quét hiệu năng. Sự kết hợp giữa một mô tả cấu trúc và một phép đo chi phí định lượng chính là cái cho phép nghiên cứu tách chi phí thời gian chạy của khả năng tương tác khỏi chi phí thực sự của nó, vốn nằm ở thiết kế API và việc học của con người.



Hình 3.2: Phương pháp tiêm lỗi có kiểm soát cho RQ2. Một danh mục cố định gồm các lớp lỗi tiêu biểu được đối chiếu trong hai ngôn ngữ: mỗi khiếm khuyết C++ được dựng và chạy dưới AddressSanitizer và ThreadSanitizer để ghi nhận giai đoạn phát hiện, trong khi cấu trúc Rust an toàn tương tự được biên dịch để ghi nhận kết quả của trình biên dịch. Vì chỉ ngôn ngữ thay đổi trong khi lỗi được tiêm được giữ cố định, khác biệt về kết quả quy được cho các bảo đảm của ngôn ngữ.

3.4 Tính công bằng như một chiến lược biến-được-kiểm-soát

Vì mỗi câu hỏi nghiên cứu quy một khác biệt cho ngôn ngữ, tính hợp lệ của toàn bộ nghiên cứu phụ thuộc vào việc giữ mọi thứ khác không đổi, và tính công bằng được xem như biến kiểm soát thí nghiệm trung tâm của nghiên cứu chứ không phải một suy nghĩ thêm về sau. Nó được cưỡng chế ở nhiều mức. *Đặc tả* được đóng băng trước khi viết bất kỳ dòng mã nào và cố định API công khai, máy trạng thái, các cấu trúc dữ liệu cùng chiến lược thuật toán của chúng, và mô hình phân mảnh cùng tương tranh (Mục 6.1). *Nền tảng thời gian chạy* được cân bằng bằng cách liên kết cùng một bộ cấp phát bộ nhớ ở cả hai bên và khớp các mức tối ưu hoá biên dịch càng sát càng tốt trong khả năng của hai bộ công cụ. *Khối lượng công việc* được tạo bởi một bộ điều khiển giống hệt nhau về cấu trúc qua các ngôn ngữ và, với một hạt giống (seed) và một quần thể UE đang hoạt động cho trước, điều khiển cả hai bản hiện thực qua đúng cùng một chuỗi thao tác, sao cho bất kỳ khác biệt độ trễ nào cũng phản ánh cách mỗi ngôn ngữ thực thi cùng một công việc chứ không phải một khác biệt trong công việc được làm. Ở những chỗ hai ngôn ngữ thực sự không thể biểu đạt cùng một cấu trúc, sự sai lệch không bị che giấu mà được ghi vào một nhật

ký thành-ngữ-bắt-buộc để có thể được cân nhắc tường minh trong phần thảo luận. Cái còn lại không được kiểm soát — chủ yếu là sự bất đối xứng kỹ năng của một tác giả duy nhất — được thừa nhận là một mối đe doạ đối với tính hợp lệ (Chương 8) chứ không tuyên bố là đã được loại bỏ.

3.5 Xử lý thống kê các phép đo

Các phép đo hiệu năng là phi tất định, nên các con số một-lần-chạy được tránh xuyên suốt. Mỗi cấu hình (ngôn ngữ × bộ cấp phát × mức tải) được đo qua nhiều lần lặp lại độc lập, và bốn biến thể được *đan xen* (interleave) trong mỗi lần lặp sao cho sự trôi chậm, các hiệu ứng nhiệt, và hoạt động nền nhiễu loạn chúng như nhau chứ không thiên lệch một biến thể. Kết quả được tóm tắt bằng trung vị với một khoảng tin cậy bootstrap phi tham số 95% thay vì bằng giá trị trung bình với một độ lệch chuẩn, vì các phân bố độ trễ có đuôi nặng và lệch, và một khoảng không-giả-định-phân-bố trên trung vị không đặt giả định chuẩn (normality). Điều này tuân theo hướng dẫn đã được thiết lập về đánh giá hiệu năng chặt chẽ về mặt thống kê, vốn cảnh báo rằng việc báo cáo một con số duy nhất mời gọi những kết luận mà biến thiên giữa các lần chạy không ủng hộ [16]. Các phân vị đuôi (p99, p99.9, p99.99) được đọc từ các biểu đồ HDR, mà việc chia thùng theo lô-ga-rit giữ được ba chữ số có nghĩa về độ phân giải trên toàn dải sao cho các phân vị cao không bị lượng tử hoá mất.

3.6 Tính tái lập và các sản phẩm nghiên cứu

Nghiên cứu được thiết kế sao cho các kết quả của nó có thể được tái tạo bởi một bên thứ ba mà không cần truy cập vào mã độc quyền của Nokia. Các bản hiện thực được điều khiển hoàn toàn bởi đặc tả công khai đã đóng băng; bộ khung đo, danh mục lỗi cùng bộ chạy của nó, và nguyên mẫu FFI được công bố cùng với luận văn; và mọi hình và bảng trong phần đánh giá đều được tạo bởi một kịch bản duy nhất trực tiếp từ các nhật ký đo thô, sao cho việc chạy lại chiến dịch trên một máy khác tái tạo các sản phẩm y nguyên. Môi trường thực thi (kiểu CPU, nhân, phiên bản trình biên dịch và bộ công cụ, và các thiết lập tần số cùng lập lịch của testbed) được ghi nhận tự động bên cạnh mỗi tập kết quả, sao cho các điều kiện của một phép đo đi cùng với dữ liệu của nó. Việc tách các kết quả nhạy-môi-trường (phép quét hiệu năng) khỏi các kết quả không-nhạy-môi-trường (kiểm thử tương đương, tiêm lỗi, và cấu trúc FFI) chính là cái cho phép luận văn gắn nhãn nhóm đầu là sơ bộ và nhóm sau là cuối cùng mà không mập mờ.

3.7 Các cân nhắc về đạo đức và bảo vệ dữ liệu

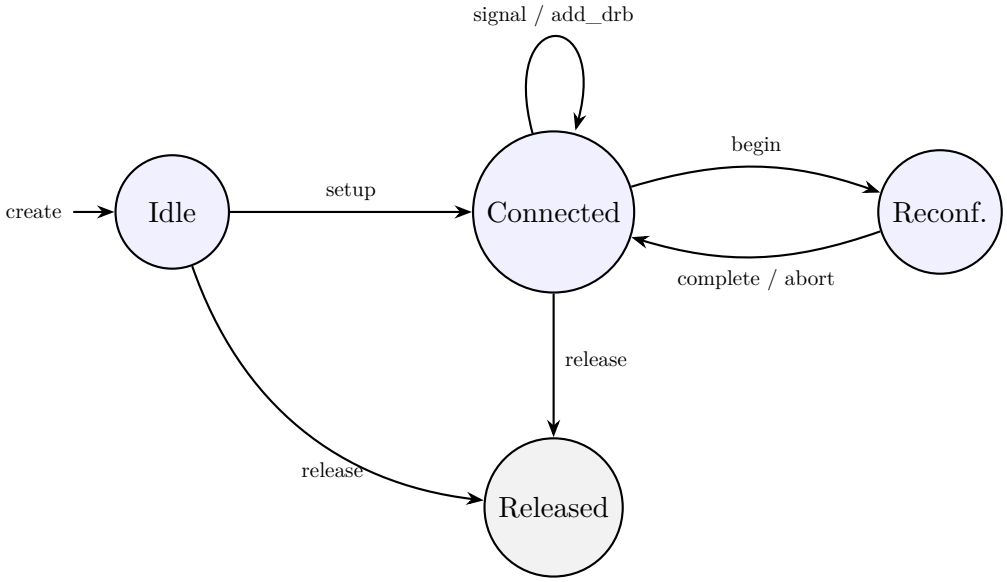
Nghiên cứu không xử lý dữ liệu cá nhân, thuê bao hay dữ liệu bảo mật của nhà khai thác nào. Toàn bộ lưu lượng là tổng hợp và được sinh bởi bộ điều khiển tất định, các ngữ cảnh UE chỉ chứa các định danh được bịa ra và một trường bảo mật giả cố định, và phân hệ được mô hình hoá theo, chứ không phải sao chép từ, mã sản xuất. Do đó không có mối lo ngại nào về quyền riêng tư, sự đồng thuận, hay lưu giữ dữ liệu, và các sản phẩm thu được có thể được phát hành công khai để tái tạo.

4. Bối cảnh bài toán: Quản lý Ngữ cảnh UE và tính dự đoán được

4.1 Phân hệ Quản lý Ngữ cảnh UE

Phân hệ được nghiên cứu là một dịch vụ Quản lý Ngữ cảnh UE, được mô hình hoá theo một miền dịch vụ gNB-CU-CP thực tế của Nokia — miền sở hữu vòng đời ngữ cảnh UE, việc thực thi thủ tục RRC [4], và tính di động/chuyển giao — chính là những chức năng mà kiến trúc 3GPP giao cho phần mặt điều khiển của một gNB đã phân rã, nơi CU-CP chứa RRC và phần mặt điều khiển của PDCP và quản lý ngữ cảnh UE, uỷ thác việc xử lý mang (bearer) mặt người dùng cho CU-UP qua giao diện E1 [1, 3, 2]. Nó là một đại diện tốt vì nó là chức năng RAN L3 cốt lõi, nhạy hiệu năng và độ trễ, vốn có trạng thái và tương tranh, và đủ tự-chứa để đặc tả chính xác và đo hiệu năng một cách công bằng. Chức năng được mô hình hoá bao gồm việc tạo và xoá ngữ cảnh UE, các chuyển trạng thái (ví dụ từ idle sang connected), xử lý thông điệp báo hiệu, và tra cứu ngữ cảnh đồng thời dưới tải. Hình 4.1 cho thấy máy trạng thái vòng đời ngữ cảnh UE mà cả hai bản hiện thực mã hoá giống hệt nhau; các chuyển đổi bất hợp lệ (ví dụ báo hiệu một ngữ cảnh `Idle`) trả về một lỗi có kiểu thay vì biến đổi trạng thái, và máy trạng thái này chính là cái mà khối lượng công việc hỗn hợp của Mục 7.2 dẫn mỗi UE đi qua.

Tính đại diện và tính khả xử lý. Hai thuộc tính khiến UECM là một đối tượng nghiên cứu tốt cho một phép so sánh ngôn ngữ. Nó *đại diện*: trạng thái khả biến theo từng UE, một vòng đời thao tác nghiêm ngặt, mức tương tranh cao giữa các UE, và các mục tiêu ngặt nghèo về độ trễ và độ tin cậy mà hệ thống 5G đặt lên các thủ tục mặt điều khiển [5] đúng là những thuộc tính khiến mã mặt điều khiển RAN sản xuất trở nên khó, nên các kết luận về cách một ngôn ngữ xử lý chúng có thể chuyển sang lớp rộng hơn gồm các dịch vụ mặt điều khiển có trạng thái, tương tranh. Nó cũng *khả xử lý*: phân hệ đủ nhỏ để đặc tả chính xác, hiện thực trung thực trong hai ngôn ngữ, và lập luận đầy đủ, chính là điều làm cho một phép so sánh có kiểm soát, đã kiểm thử tương đương trở nên khả thi. Một phân hệ lớn hơn hoặc kém giới



Hình 4.1: Máy trạng thái vòng đời ngữ cảnh UE. `create` cấp phát một ngữ cảnh ở trạng thái `Idle`; `setup` kết nối nó; `signal` và `add_drb` được xử lý tại chỗ khi đang `Connected`; một lần cấu hình lại/chuyển giao đi qua `Reconfiguring` rồi quay lại khi `complete` hoặc `abort`; `release` tháo dỡ ngữ cảnh. Các thao tác bất hợp lệ cho trạng thái hiện tại trả về một `OpError` thay vì thay đổi trạng thái.

hạn hơn sẽ cải thiện tính hợp lệ bên ngoài với cái giá là tính hợp lệ bên trong mà một phép so sánh ngôn ngữ công bằng phụ thuộc vào; UECM được chọn như điểm giữ được cả hai trong tầm với.

Tập thao tác. Giao diện được nghiên cứu vận dụng được thiết kế cố ý nhỏ và hướng-vòng-đời: tạo và giải phóng một ngữ cảnh; thiết lập một kết nối (`Idle`→`Connected`); xử lý một thông điệp báo hiệu trong phiên; thêm một mang dữ liệu vô tuyến (data radio bearer); bắt đầu, hoàn tất, hoặc huỷ bỏ một lần cấu hình lại/chuyển giao; và tra cứu trạng thái của một ngữ cảnh dưới tải đồng thời. Mỗi thao tác là *toàn phần* (total) đối với máy trạng thái ở Hình 4.1: một thao tác bất hợp lệ ở trạng thái hiện tại trả về một lỗi có kiểu và để ngữ cảnh nguyên vẹn, thay vì ném ngoại lệ hay làm hỏng trạng thái. Tập thao tác đóng, được định kiểu chặt chẽ này chính là cái cho phép cùng một bộ vector kiểm thử và cùng một khối lượng công việc điều khiển cả hai bản hiện thực đi theo các đường giống hệt nhau (Mục 6.7).

4.2 Các lớp lỗi an toàn bộ nhớ và tương tranh

Các khiếm khuyết gây hại cho mã mặt điều khiển C++ loại này rơi vào hai họ. Họ *an toàn bộ nhớ* liên quan đến vòng đời và biên của các đối tượng: một lỗi *sử-dụng-sau-khi-giải-phóng* truy cập vùng nhớ đã được giải phóng, một lỗi *giải-phóng-hai-lần*

giải phóng cùng một vùng nhớ hai lần, và một lỗi *tràn bộ đệm* đọc hoặc ghi vượt quá cuối một vùng cấp phát. Họ *tương tranh* liên quan đến truy cập không đồng bộ hoá hoặc thứ tự sai từ nhiều luồng: một *tranh chấp dữ liệu* là một cặp truy cập không đồng bộ hoá tới cùng một vị trí mà ít nhất một là ghi, một *deadlock* là một sự chờ khoá theo vòng, và *vô hiệu hoá iterator* là việc dùng một iterator hoặc tham chiếu vào một bộ chứa sau khi một biến đổi đã di chuyển hoặc giải phóng vùng nhớ nền. Điểm chung của chúng là tất cả đều *im lặng* trong C++: mỗi lỗi được trình biên dịch chấp nhận và, trên các đường mà các kiểm thử thông thường tình cờ thực thi, có thể có vẻ hoạt động, chỉ nổi lên về sau dưới dạng hỏng dữ liệu, một cú sập, hay một đột biến độ trễ khó lường — chính là những lớp hỏng-bộ-nhớ mà nhiều thập kỷ nghiên cứu bảo mật đã chật vật kiềm chế trong C và C++ [30].

Mỗi lớp có một hiện thân tự nhiên trong Quản lý Ngữ cảnh UE, đó là lý do vì sao phân hệ này là một phương tiện tốt để nghiên cứu chúng:

- **Sử-dụng-sau-khi-giải-phóng.** Một thông điệp báo hiệu giữ một con trỏ hoặc tham chiếu thô tới một ngữ cảnh UE mà một lệnh `release` đồng thời đã xoá khỏi bản đồ ngữ cảnh; bộ xử lý đang trong luồng sau đó truy xuất vùng nhớ đã giải phóng.

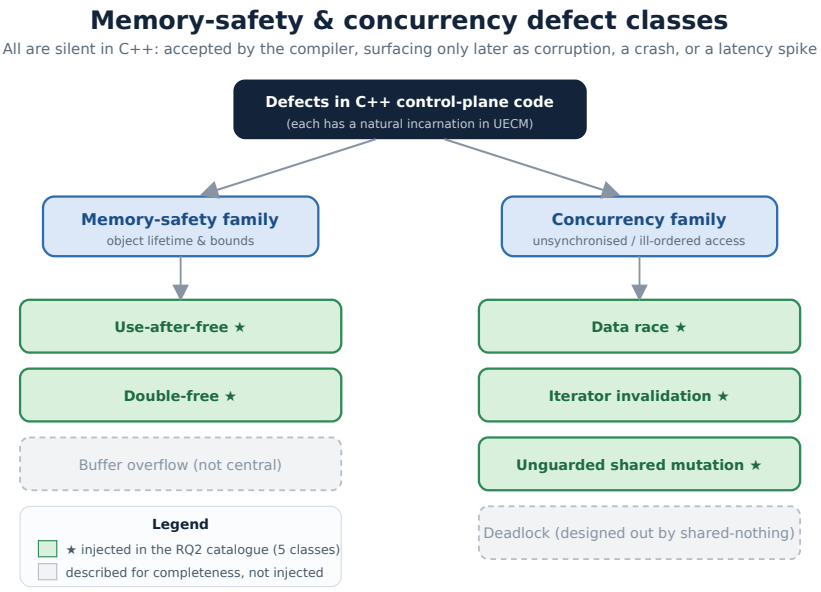
- **Giải-phóng-hai-lần.** Một đường release chạy hai lần cho cùng một UE, ví dụ vì một lệnh release được truyền lại không được làm cho lũy đẳng (idempotent), giải phóng các tài nguyên do ngữ cảnh sở hữu một lần thứ hai.

- **Vô hiệu hoá iterator.** Mã lặp qua danh sách mang dữ liệu vô tuyến của một ngữ cảnh trong khi thêm hoặc bớt một mang, nên iterator trỏ vào vùng nhớ mà bộ chứa đã cấp phát lại từ đó.

- **Tranh chấp dữ liệu.** Hai worker cập nhật cùng một trường của ngữ cảnh — một bộ đếm hoạt động hoặc một cờ trạng thái — mà không đồng bộ hoá, nên các truy cập tranh chấp nhau.

- **Biến đổi chia sẻ không được bảo vệ.** Một ngữ cảnh được chia sẻ bằng tham chiếu qua các luồng và bị biến đổi mà không có khoá hay kỷ luật khác, một tranh chấp mà nguyên nhân gốc là sự vắng mặt của bất kỳ quy tắc sở hữu nào cho trạng thái chia sẻ.

Năm lớp này chính là danh mục được tiêm vào trong nghiên cứu tiêm lỗi của Chương 7: chúng vừa xuất hiện tự nhiên trong phân hệ này vừa nằm gọn trong các bảo đảm mà Rust tuyên bố cung cấp, điều này khiến chúng là phép kiểm tra công bằng nhất về việc liệu tuyên bố đó có đúng hay không. Deadlock và tràn bộ đệm được mô tả ở đây cho đầy đủ nhưng không phải trung tâm của danh mục được tiêm,

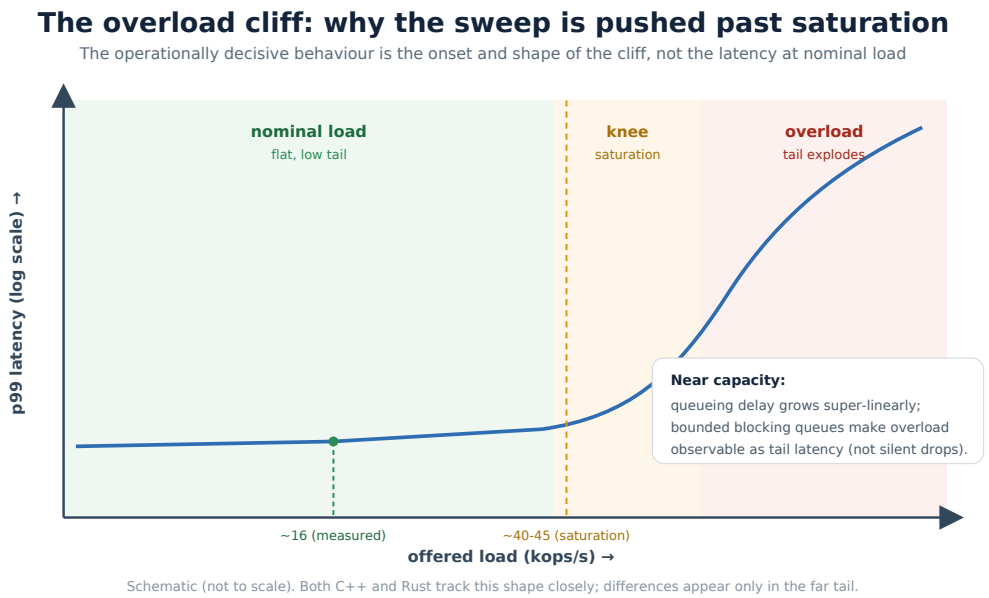


Hình 4.2: Các lớp lỗi gây hại cho mã mặt điều khiển C++, được chia thành một họ an toàn bộ nhớ (vòng đời và biên của đối tượng) và một họ tương tranh (truy cập không đồng bộ hoá hoặc sai thứ tự). Năm lớp có dấu sao là những lớp được tiêm trong nghiên cứu tiêm lỗi RQ2; tràn bộ đệm và deadlock được mô tả cho đầy đủ nhưng không phải trung tâm của danh mục. Tất cả đều im lặng trong C++, chỉ nổi lên về sau dưới dạng hỏng dữ liệu, một cú sập, hay một đột biến độ trễ.

cái trước vì thiết kế không-chia-sẻ-gì (Chương 6) phần lớn đã thiết kế loại bỏ các khoá khỏi đường nóng (hot path), và cái sau vì không bản hiện thực nào thực hiện số học con trỏ thô trên kho ngữ cảnh.

4.3 Độ trễ đuôi và tính dự đoán được trong RAN

Đối với các thủ tục RAN bị dẫn dắt bởi hạn chót, câu hỏi hiệu năng có ý nghĩa là các trường hợp chậm tệ đến đâu, chứ không phải trung bình tốt đến đâu. Đây là bài toán "đuôi ở quy mô lớn" (tail at scale) kinh điển: trong một hệ thống nơi một hành động mà người dùng thấy được phân nhánh thành nhiều thao tác phụ thuộc, thành phần chậm nhất chi phối trải nghiệm đầu-cuối, nên phần đuôi của phân bố độ trễ, chứ không phải giá trị trung bình của nó, quyết định liệu các hạn chót có được đáp ứng hay không [12]. Thang thời gian liên quan là micro-giây: xử lý I/O độ-trễ-thấp và xử lý yêu cầu trong-bộ-nhớ hiện đại vận hành trong một chế độ mà các hệ thống hiện có, được tinh chỉnh cho các sự kiện cỡ nano-giây và mili-giây, hỗ trợ kém [9], điều này khiến việc đo mức micro-giây một cách kỷ luật trở nên thiết yếu — càng vậy vì phần lớn phần đuôi bắt nguồn bên dưới ứng dụng, trong các hiệu ứng phần cứng, hệ điều hành và lập lịch [21]. Do đó nghiên cứu đặc trưng hoá toàn bộ phân bố độ trễ, tập trung vào các phân vị cao (p99, p99.9, p99.99), giá trị lớn nhất, và độ



Hình 4.3: Vách quá tải (sơ đồ minh hoạ). Độ trễ đuôi giữ ở mức thấp và bằng phẳng khi tải đưa vào còn dưới năng lực phục vụ, rồi leo dốc đứng một khi hệ thống bị đẩy vượt qua bão hoà và các yêu cầu xếp hàng sau các hàng đợi có chặn, chặn nghẽn. Hành vi mang tính quyết định về mặt vận hành là sự khởi phát và hình dạng của vách này, đó là lý do vì sao phép quét tải được đẩy vượt qua điểm bão hoà thay vì dừng lại ở tải danh định.

dao động, cùng cách chúng suy giảm khi hệ thống tiến gần và vượt qua bão hoà. Đo những thứ này cho đúng đòi hỏi phải cẩn trọng để tránh thiên lệch bỏ-sót-phối-hợp mà các vòng lặp định thời ngây thơ gây ra khi hệ thống khựng lại [31].

Hạn chót, phân nhánh và quá tải. Hai đặc điểm của bối cảnh RAN biến độ trễ đuôi từ một sự tinh tế hàn lâm thành thước đo chi phối. Đặc điểm thứ nhất là *phân nhánh* (fan-out): một hành động mà người dùng thấy được — một lần thiết lập cuộc gọi hay một cuộc chuyển giao — phân rã thành nhiều thao tác mặt điều khiển phụ thuộc, và vì thao tác chậm nhất trong số đó quyết định thời điểm hoàn thành, một độ trễ hiếm gặp ở mỗi thao tác trở thành phổ biến ở mỗi thủ tục [12]. Đặc điểm thứ hai là *quy mô*: một trạm bận thực hiện hàng nghìn thủ tục như vậy mỗi giây trên hàng triệu thiết bị kết nối, nên một sự kiện đuôi xảy ra một lần trong một nghìn thao tác không phải là trường hợp biên mà là chuyện thường xuyên xảy ra ở đâu đó trong đội tại mọi thời điểm. Làm trầm trọng cả hai là hành vi xếp hàng gần năng lực: khi tải đưa vào tiến gần tốc độ phục vụ, trễ xếp hàng tăng siêu tuyến tính và phần đuôi bùng nổ, đó là lý do vì sao một sự đặc trưng hoá dừng lại ở tải danh định bỏ lỡ chính cái chế độ thực sự quyết định chất lượng dịch vụ. Đây là lập luận đằng sau việc quét khối lượng công việc vào trong quá tải và báo cáo toàn bộ phân bố độ trễ thay vì giá trị trung bình (Chương 7).

5. Mục tiêu

5.1 Những thiếu sót trong các phép so sánh hiện có

Các nghiên cứu đa ngôn ngữ báo cáo rằng các ngôn ngữ hệ thống được biên dịch như C, C++ và Rust đạt được hiệu quả về thời gian và năng lượng nhìn chung tương tự nhau, vượt xa các ngôn ngữ có runtime quản lý và thông dịch [26]. Bằng chứng gần nhất trong bối cảnh mạng đến từ xử lý gói ở không gian người dùng: một trình điều khiển mạng 10 Gbit/s viết bằng Rust theo lối thành ngữ thực thi nhiều lệnh trên mỗi gói hơn nhiều so với bản C tham chiếu của nó nhưng chỉ chạy chậm hơn khoảng 4% [13], gợi ý rằng các kiểm tra an toàn của Rust phần lớn được các CPU thực-thi-ngoài-thứ-tự (out-of-order) hiện đại hấp thụ. Tuy nhiên, các phép so sánh này phần lớn được thiết lập trên các phép đo đa dụng hoặc chuyển tiếp gói; chúng nhấn mạnh hiệu năng trung bình hoặc thông lượng, hiếm khi khảo sát độ trễ đuôi khi quá tải cho các khối lượng công việc mặt điều khiển có trạng thái, tương tranh, không cô lập các lớp lỗi mà một ngôn ngữ ngăn chặn, và ít khi định lượng chi phí di trú tăng dần vào một mã nguồn C++ sẵn có — một chi phí mà các nghiên cứu áp dụng định tính báo cáo là bị chi phối bởi đường cong học tập chứ không phải bởi chi phí thời gian chạy [15]. Đây chính là những khoảng trống mà luận văn này nhắm tới.

5.2 Các mục tiêu giải pháp

Mục tiêu bao trùm là một phép so sánh công bằng, tái lập được giữa C++ hiện đại và Rust cho một phân hệ mặt điều khiển RAN, cùng với các sản phẩm tái sử dụng được và hướng dẫn dựa trên bằng chứng. Nó phân rã thành các mục tiêu cụ thể, mỗi mục tiêu gắn với một câu hỏi nghiên cứu sao cho tiến độ có thể truy vết:

O1 (tính dự đoán được, RQ1) Định lượng và so sánh các phân bố độ trễ của hai bản hiện thực, tập trung vào phần đuôi p99/p99.9 và độ dao động, qua một

phép quét tải đạt tới quá tải, và xác định ở đâu dọc theo phép quét đó, nếu có, hai ngôn ngữ tách nhau.

O2 (ngăn chặn lỗi, RQ2) Xác định, đối với một danh mục các lớp lỗi tiêu biểu cho phân hệ, những lớp nào được ngăn chặn tại thời điểm biên dịch trong Rust an toàn và, đối với những lớp không được ngăn, chúng được phát hiện ở giai đoạn nào và với chi phí bao nhiêu trong C++.

O3 (chi phí phụ trội, RQ3) Đo chi phí tài nguyên — thông lượng, mức dùng CPU, và hành vi bộ nhớ — của mỗi bản hiện thực dưới các điều kiện khớp nhau, cô lập bất kỳ hình phạt hệ thống nào quy được cho mô hình an toàn của Rust.

O4 (di trú, RQ4) Xây dựng và đo một nguyên mẫu áp-dụng-tăng-dần thực tế, định lượng kích thước của đường ghép tương tác, phạm vi `unsafe` mà nó đòi hỏi, và chi phí mỗi lời gọi của việc băng qua ranh giới ngôn ngữ.

O5 (sản phẩm và hướng dẫn) Cung cấp một đặc tả đã đóng băng, một cặp bản hiện thực đã kiểm thử tương đương, một khung đo hiệu năng tái lập được, một phân loại các lớp lỗi, và các hướng dẫn kỹ thuật mà những người hành nghề có thể tái sử dụng cho các quyết định lựa chọn ngôn ngữ trong quá trình tiến hoá RAN gốc-đám-mây.

5.3 Các giả thuyết

Các mục tiêu được tiếp cận với ba kỳ vọng tiên nghiệm, được phát biểu sao cho phần đánh giá có thể xác nhận hoặc bác bỏ chúng chứ không chỉ mô tả kết quả sau khi sự việc đã rồi. **H1:** đối với khối lượng công việc này, hai ngôn ngữ tương đương về hiệu năng trong vùng liên quan về mặt vận hành nằm dưới bão hoà, với bất kỳ khác biệt nào cũng chỉ giới hạn ở phần đuôi xa. **H2:** Rust an toàn ngăn chặn tại thời điểm biên dịch các lớp lỗi an toàn bộ nhớ và tương tranh mà C++ để cho việc phát hiện nỗ-lực-tối-đa, sau-khi-sự-việc-đã-rồi. **H3:** chi phí thời gian chạy của khả năng tương tác C++-sang-Rust là không đáng kể so với chi phí của các thao tác được gọi, nên chi phí thực sự của việc áp dụng tăng dần là công sức kỹ thuật chứ không phải hiệu năng. Các giả thuyết này được xem xét lại đối chiếu với bằng chứng trong Chương 8.

5.4 Các tiêu chí thành công

Nghiên cứu thành công tới mức nó tạo ra bằng chứng *đủ-để-ra-quyết-định* — bằng chứng mà một người hành nghề có thể dựa vào để lựa chọn ngôn ngữ — chứ không

chỉ là một tập các phép đo. Cụ thể điều này đòi hỏi: mọi phép so sánh hiệu năng phải được báo cáo với các lần lặp lại và các khoảng tin cậy sao cho một khác biệt được tuyên bố có thể phân biệt được với nhiễu giữa các lần chạy; kết quả ngăn-chặn-lỗi phải được chứng minh trên các chương trình cụ thể, biên dịch được chứ không phải lập luận chỉ từ ngữ nghĩa ngôn ngữ; chi phí di trú phải được đọc ra từ một nguyên mẫu hoạt động được chứ không phải ước lượng; và toàn bộ phải tái lập được từ các sản phẩm được công bố và một đặc tả công khai. Đáp ứng các tiêu chí này chính là cái phân biệt đóng góp này với các phép so sánh đa dụng, trường hợp trung bình mà những thiếu sót của chúng là động lực cho công trình.

5.5 Phạm vi và ranh giới

Công trình tập trung vào một phân hệ tiêu biểu duy nhất được nghiên cứu dưới các điều kiện tổng hợp, có kiểm soát, và các ranh giới của nó được vạch ra một cách có chủ đích để bảo vệ tính hợp lệ bên trong của phép so sánh ngôn ngữ. Trong phạm vi là chức năng Quản lý Ngữ cảnh UE của Chương 4, thiết kế tương tranh không-chia-sẻ-gì, phép quét tải đến quá tải, danh mục tiêm lỗi, và nguyên mẫu FFI. Nằm ngoài phạm vi một cách tường minh là toàn bộ hệ thống gNB và các miền mặt điều khiển còn lại của nó, xử lý PHY/L2 và mặt người dùng nặng-dữ-liệu, các vết lưu lượng thực được ghi lại (đối lập với một khối lượng công việc tổng hợp được biện minh theo đặc tả), các phương án ngôn ngữ có runtime quản lý (bị loại trừ trên cơ sở độ trễ đuôi đã lập luận trong Chương 2), và việc triển khai sản xuất cùng các mối quan tâm vận hành như nâng cấp, khả năng quan sát (observability), và đa-người-thuê (multi-tenancy). Các loại trừ này đánh đổi tính hợp lệ bên ngoài để lấy các điều kiện có kiểm soát mà một phép so sánh ngôn ngữ công bằng cần, và mỗi loại trừ được xem xét lại như một giới hạn trong Chương 8.

6. Thiết kế và Hiện thực

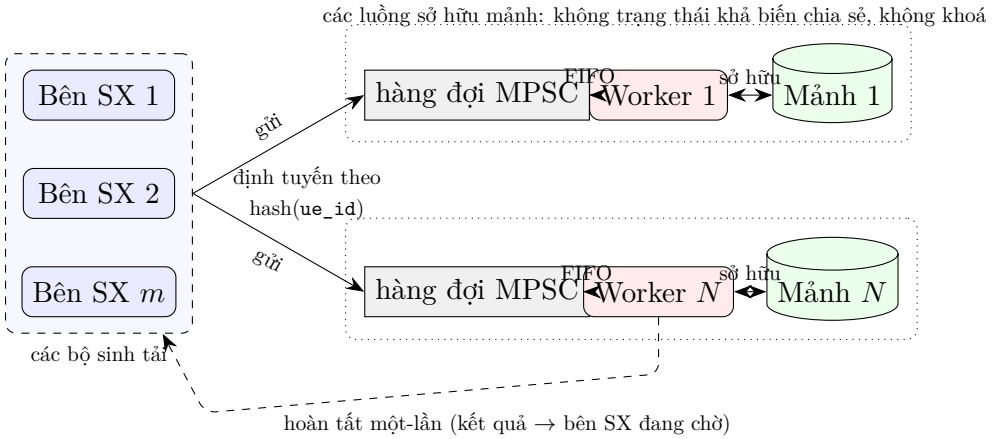
6.1 Đặc tả chung: đường cơ sở công bằng

Trước khi viết bất kỳ dòng mã nào, một đặc tả độc lập ngôn ngữ duy nhất được đóng băng: API công khai, máy trạng thái UE, các cấu trúc dữ liệu cùng chiến lược thuật toán của chúng (ví dụ một bản đồ băm phân mảnh được khoá theo định danh UE), độ mịn của việc khoá và phân mảnh, và mô hình tương tranh. Cả hai bản hiện thực tuân theo đặc tả này để phép so sánh đo các khác biệt ngôn ngữ chứ không phải sự bất đối xứng về thiết kế hay kỹ năng. Bản thân đặc tả là một sản phẩm bàn giao và là biện pháp kiểm soát chính chống lại thiên lệch hiện thực.

6.2 Mô hình khái niệm và thiết kế tương tranh

Hai bản hiện thực tương đương về chức năng được phát triển dựa trên đặc tả chung: một bản C++ hiện đại (C++23, STL, RAII, luồng chuẩn) và một bản Rust (bộ công cụ ổn định, mô hình bộ nhớ dựa trên quyền sở hữu, tương tranh an toàn). Một mô hình tương tranh duy nhất được dùng cho cả hai: một mô hình *không-chia-sẻ-gì, một-luồng-trên-mỗi-mảnh* phản chiếu cách RAN L3 thực tế phân vùng trạng thái UE, và việc trộn lẫn các mô hình bất đồng bộ (async) với dựa-trên-luồng được cố ý tránh.

Không gian định danh UE được phân vùng thành N_{shards} mảnh (mặc định là tám), mỗi mảnh được sở hữu bởi đúng một luồng worker. Chỉ worker đó đọc hoặc biến đổi các ngữ cảnh của mảnh của nó, nên không có trạng thái khả biến chia sẻ giữa các worker và không có khoá trên dữ liệu ngữ cảnh. Các bên gọi (các luồng sinh tải) không bao giờ chạm trực tiếp vào trạng thái ngữ cảnh; chúng gửi các thông điệp yêu cầu tới hàng đợi nhiều-bên-sản-xuất, một-bên-tiêu-thụ (MPSC) có chặn của một mảnh và chờ trên một handle hoàn tất dùng-một-lần mà worker ghi kết quả vào đó. Hàng đợi *có chặn và chặn nghẽn* (bounded and blocking): khi hàng đợi của một mảnh đầy, việc gửi bị chặn cho tới khi có chỗ trống, điều này tạo ra trễ xếp hàng thực tế khi quá tải và làm cho độ trễ đuôi trở thành đại lượng quan sát của nghiên

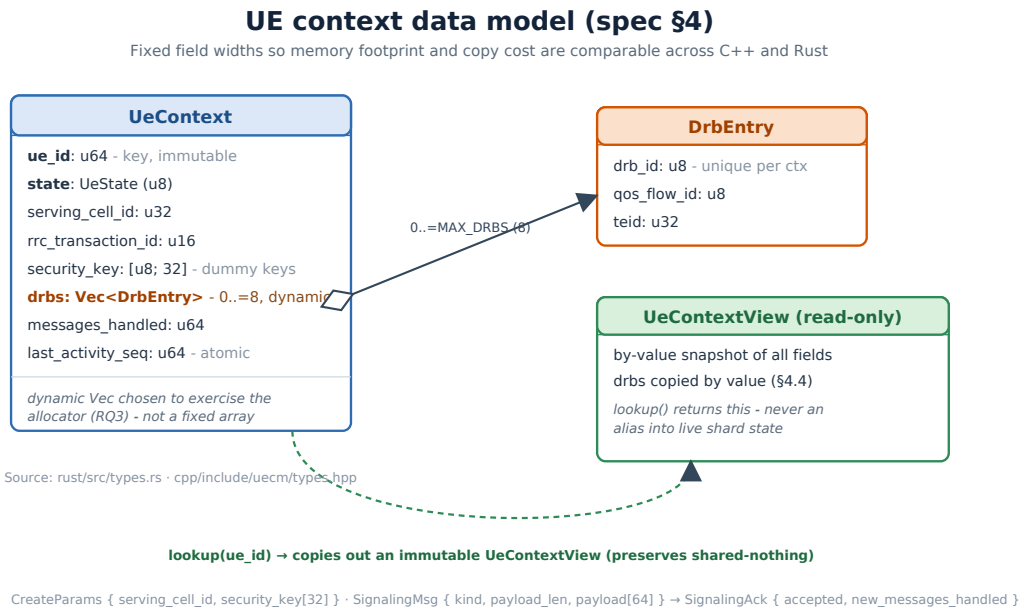


Hình 6.1: Kiến trúc không-chia-sẻ-gì, một-luồng-trên-mỗi-mảnh. Các bên sản xuất gửi các thông điệp yêu cầu tới hàng đợi MPSC có chặn, chặn nghẽn của một mảnh (được chọn bằng một hàm băm cố định của `ue_id`) và chờ trên một handle hoàn tất dùng-một-lần; mỗi mảnh được đọc và ghi bởi đúng một luồng worker, nên không có khoá trên trạng thái ngữ cảnh. Việc chặn trên một hàng đợi đầy chính là cái làm cho quá tải trở nên quan sát được dưới dạng độ trễ đuôi.

cứu (một chính sách loại-bỏ không-chặn sẽ che giấu quá tải). Các yêu cầu tới cùng một mảnh được xử lý theo đúng thứ tự vào-trước-ra-trước. Cấu trúc này chính là cái làm cho phép so sánh tương-tranh-an-toàn có ý nghĩa: bộ máy thú vị là hàng đợi, sự chuyển giao hoàn tất, và việc chuyển giao quyền sở hữu của các yêu cầu, chứ không phải sự tranh chấp khoá. Trạng thái khả biến chia sẻ duy nhất được cho phép là một bộ đếm nguyên tử (atomic) đơn dùng để đóng dấu một số thứ tự hoạt động theo từng ngữ cảnh, vốn là sổ sách chỉ-ghi không ảnh hưởng đến dòng điều khiển. Hình 6.1 cho thấy luồng dữ liệu này.

6.3 Các lựa chọn kiến trúc và ngăn xếp công nghệ

Phía C++ dùng CMake với Ninja, thư viện chuẩn, và `std::thread`; phía Rust dùng Cargo và bộ công cụ ổn định. Để giữ cho phép so sánh là một phép so sánh ngôn ngữ chứ không phải một phép so sánh bộ cấp phát, cả hai liên kết cùng một bộ cấp phát bộ nhớ: C++ liên kết `jemalloc` của hệ thống và Rust cài đặt một bộ cấp phát toàn cục `jemalloc` (từ `tikv-jemallocator`), với một bản dựng dùng-bộ-cấp-phát-hệ-thống được giữ lại ở mỗi bên làm đối chứng. Cả hai ghi nhận độ trễ qua cùng một thuật toán biểu đồ dải-động-cao (HDR) [31], được cấu hình giống hệt nhau (dải $[1\,\text{ns}, 3.6\,\text{s}]$, ba chữ số có nghĩa); Rust dùng crate `hdrhistogram` chuẩn tắc và C++ dùng một bản chuyển chỉ-header của cùng thuật toán chia thùng, được kiểm định đối chiếu với một phân bố đã biết. Việc tối ưu hoá bản dựng được khớp càng sát càng tốt trong khả năng của hai bộ công cụ: hồ sơ release của Rust được ghim ở



Hình 6.2: Mô hình dữ liệu ngữ cảnh UE được chia sẻ bởi cả hai bản hiện thực. Độ rộng các trường được cố định sao cho dấu chân bộ nhớ và chi phí sao chép có thể so sánh được giữa các ngôn ngữ; danh sách mang dữ liệu vô tuyến là một `Vec`/`std::vector` động một cách có chủ đích, để vận dụng bộ cấp phát cho RQ3. `lookup` trả về một `UeContextView` bất biến được sao chép ra theo giá trị chứ không phải một bí danh vào trạng thái mảnh đang sống, bảo toàn bảo đảm không-chia-sẻ-gì.

`opt-level = 2` để khớp với bản dựng `-O2` của C++.

6.4 Chi tiết hiện thực

Cả hai bản hiện thực tuân theo đặc tả đã đóng băng theo từng-trường một. Mỗi ngữ cảnh UE lưu định danh của nó, trạng thái vòng đời (một liệt kê bốn-giá-trị `Idle`, `Connected`, `Reconfiguring`, `Released`), ô phục vụ (serving cell), id giao dịch RRC, một ngữ cảnh bảo mật giả cố định 32 byte, một danh sách động tối đa tám mang dữ liệu vô tuyến, và hai bộ đếm. Danh sách mang dữ liệu vô tuyến được cố ý là một bộ chứa tăng-trưởng-được (`std::vector` / `Vec`) thay vì một mảng cố định, để nghiên cứu bộ cấp phát (RQ3) quan sát được việc cấp phát theo từng ngữ cảnh một cách thực tế. Các thao tác đọc trả về một ảnh chụp bất biến (một `UeContextView`) được sao chép ra khỏi worker chứ không phải một tham chiếu vào trạng thái mảnh đang sống, điều này bảo toàn bảo đảm không-chia-sẻ-gì và giữ cho hai bản hiện thực giống hệt nhau về mặt ngữ nghĩa.

Chọn mảnh và kho lưu trữ trong-mảnh. Để ngăn các bộ băm của thư viện chuẩn trở thành một lợi thế ngôn ngữ tình cờ, cả hai bên đều ghim bộ băm của mình. Việc gán mảnh áp dụng bộ hoàn tất SplitMix64 lên `ue_id` trước khi rút

gọn modulo số mảnh, tránh sự lệch (skew) cho các định danh tuần tự. Bộ chứa theo từng mảnh là bản đồ băm chuẩn của mỗi ngôn ngữ (`std::unordered_map` / `std::collections::HashMap`) nhưng được khoá qua một hàm băm FNV-1a 64-bit cố định trên tám byte little-endian của định danh; một kiểm thử kiểm tra rằng `BuildHasher` của Rust tạo ra một digest giống hệt từng-bit với bản tham chiếu `fnv1a_64`. Cả hai bản đồ đều dự trữ một dung lượng ban đầu lúc khởi động sao cho các phép đo ở trạng thái ổn định không bị nhiễu loạn bởi việc băm lại (rehashing).

Các thành ngữ bị ngôn ngữ ép buộc. Ở những chỗ hai ngôn ngữ buộc phải phân kỳ để thoả cùng một yêu cầu, sự sai lệch được ghi vào nhật ký thành-ngữ-bắt-buộc của đặc tả để có thể được cân nhắc trong phần thảo luận. Kiểu kết quả là `std::expected<T, OpError>` trong C++23 và `Result<T, OpError>` theo lối thành ngữ trong Rust. Một yêu cầu là một struct có thể chỉ-di-chuyển (move-only) mang một hoàn tất `std::promise` trong C++, và một `enum` mà mỗi biến thể mang một bộ gửi (sender) một-lần có kiểu trong Rust, nên bên gọi Rust không cần điều phối biến thể tại thời gian chạy trên đáp ứng. Hàng đợi có chặn, chặn nghẽn là một cấu trúc `mutex`-cộng-biến-điều-kiện tự viết trong C++ và một `sync_channel` trong Rust; hoàn tất một-lần là `std::promise`/`std::future` trong C++ và một kênh theo từng yêu cầu trong Rust.

Câu chuyện an toàn. Thư viện Rust *không* chứa mã `unsafe` nào: Rust an toàn bảo đảm không có tranh chấp dữ liệu tại thời điểm biên dịch thông qua các trait `Send`/`Sync`, nên không cần bộ dò lỗi thời gian chạy nào để thiết lập điều đó. Bản hiện thực C++ không thực hiện việc quản lý bộ nhớ thủ công nào ngoài các bộ chứa STL và đã được kiểm chứng là không có tranh chấp dữ liệu dưới ThreadSanitizer. Sự bất đối xứng này — một bảo đảm tại thời điểm biên dịch ở một bên so với một kiểm thử được đo lường ở bên kia — chính là cái mà nghiên cứu tiêm lỗi của Chương 7 định lượng.

6.5 API công khai và ngữ nghĩa thao tác

Cả hai bản hiện thực phơi bày cùng một API nhỏ, hướng-vòng-đời dựa trên đặc tả chung, sao cho bề mặt mà một bên gọi thấy là giống hệt nhau tới mức các quy ước đặt tên của mỗi ngôn ngữ. Một dịch vụ được tạo với một số mảnh cố định và một giới hạn hàng đợi theo từng mảnh; sau đó các bên gọi gọi các thao tác ngữ cảnh — create, setup, handle-signaling, add-bearer, reconfigure begin/complete/abort, release, và tra cứu trạng thái — mỗi thao tác được nhận diện bởi `ue_id` đích. Mọi thao tác đều *toàn phần* và *có kiểu*: nó trả về một giá trị thành công hoặc một `OpError` có kiểu rút

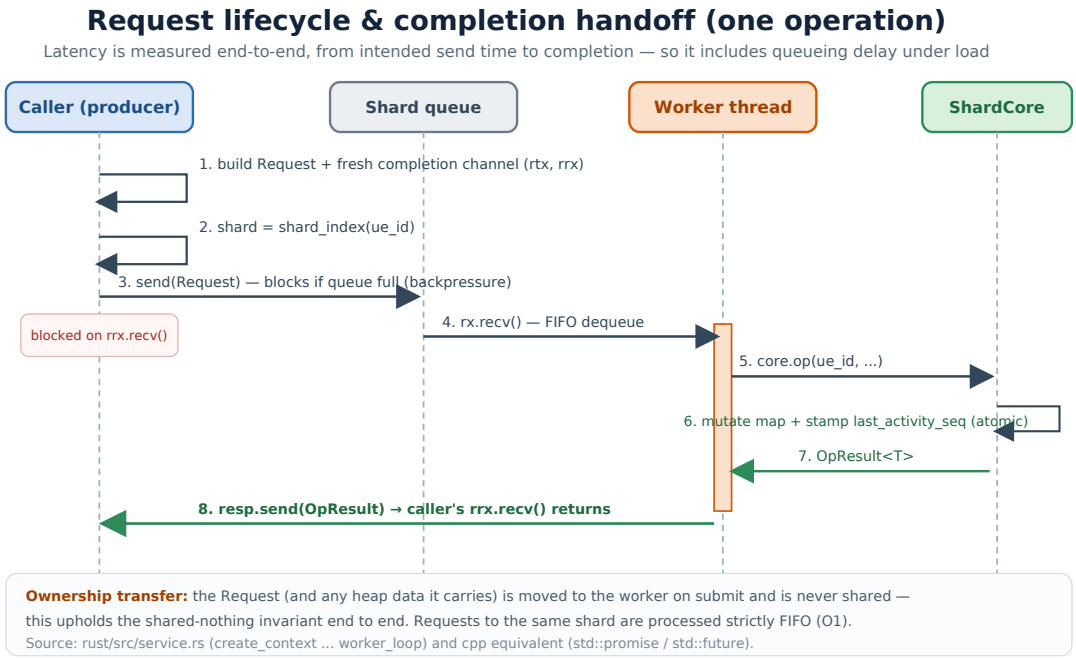
ra từ một liệt kê đóng (ví dụ `NotFound`, `InvalidState`, `BearerLimit`), và một thao tác bất hợp lệ đối với trạng thái hiện tại của ngữ cảnh (Hình 4.1) trả về lỗi tương ứng và để ngữ cảnh nguyên vẹn chứ không biến đổi nó hay báo hiệu thất bại ngoài luồng. Lựa chọn này quan trọng cho tính công bằng ngang với cho tính đúng đắn: vì không bản hiện thực nào dùng ngoại lệ hay mã lỗi như một kênh dòng-điều-khiển ẩn, chi phí và dòng điều khiển của đường thất bại đều hiển thị và giống hệt nhau ở cả hai bên, và cùng một bộ vector tuân thủ (Mục 6.7) có thể khẳng định sự nhất trí từng-thao-tác trên cả giá trị trả về lẫn trạng thái kết quả.

6.6 Vòng đời yêu cầu và sự chuyển giao hoàn tất

Đường đi đầu-cuối của một thao tác đơn lẻ là bộ máy mà nghiên cứu thực sự đo, nên đáng để phát biểu chính xác. Một bên gọi dựng một yêu cầu mang thao tác, các tham số của nó, và một handle hoàn tất dùng-một-lần, rồi tính mảnh đích từ một hàm băm cố định của `ue_id`. Nó gửi yêu cầu tới hàng đợi MPSC có chặn của mảnh đó: nếu hàng đợi còn chỗ, yêu cầu được xếp vào và bên gọi chặn trên handle hoàn tất; nếu hàng đợi đầy, *việc gửi* bị chặn cho tới khi có chỗ trống, đây là điểm mà quá tải đưa vào trở nên quan sát được dưới dạng trễ xếp hàng thay vì bị loại bỏ âm thầm. Worker đơn của mảnh lấy các yêu cầu ra theo đúng thứ tự FIFO, thực thi thao tác trên kho mà nó sở hữu độc quyền, ghi kết quả (một giá trị thành công hoặc một `OpError`, và với các thao tác đọc là một `UeContextView` bất biến được sao chép ra khỏi trạng thái đang sống) vào handle hoàn tất, rồi chuyển sang yêu cầu tiếp theo. Quyền sở hữu của yêu cầu, và của bất kỳ dữ liệu heap nào nó mang, được chuyển giao cho worker khi gửi và không bao giờ được chia sẻ, chính là cái giữ vững bất biến không-chia-sẻ-gì từ đầu đến cuối. Độ trễ được đo trên toàn bộ đường đi này, từ thời điểm gửi dự định tới lúc hoàn tất, sao cho nó bao gồm trễ xếp hàng dưới tải và phản ánh cùng một vòng khứ hồi xuyên-luồng ở cả hai bên.

6.7 Kiểm chứng tương đương

Vì toàn bộ phép so sánh dựa trên việc hai bản giống hệt nhau về chức năng, tính tương đương được kiểm thử chứ không phải giả định. Một bộ kiểm thử tuân thủ trung-lập-ngôn-ngữ biểu diễn các vector kiểm thử dưới dạng các chuỗi thao tác cùng kết quả kỳ vọng và một bản tóm tắt hậu-trạng-thái kỳ vọng; một bộ chạy trong mỗi ngôn ngữ thực thi các vector và phát ra một bản ghi chuẩn tắc, rồi hai bản ghi được so khác (diff) để kiểm tra sự nhất trí từng-thao-tác. Bộ kiểm thử có hạt giống phủ toàn bộ mô hình lỗi, các đường cấu hình lại begin/complete/abort, việc lấp đầy và



Hình 6.3: Đường đi đầu-cuối của một thao tác đơn lẻ và sự chuyển giao hoàn tất. Bên gọi dựng một yêu cầu với một handle hoàn tất dùng-một-lần, định tuyến nó bằng `shard_index(ue_id)` tới hàng đợi có chặn của mảnh sở hữu (chặn nếu nó đầy), và chờ; worker đơn của mảnh lấy ra theo thứ tự FIFO, thực thi thao tác trên kho nó sở hữu độc quyền, và ghi kết quả trở lại handle hoàn tất. Quyền sở hữu của yêu cầu được chuyển giao cho worker khi gửi và không bao giờ được chia sẻ. Độ trễ được đo trên toàn bộ đường đi.

tràn mang dữ liệu vô tuyến, và tính tất định của việc gán mảnh; cả hai bản hiện thực hiện đều vượt qua và nhất trí. Các khối lượng công việc đo hiệu năng bổ sung một tín hiệu tương đương thứ hai, độc lập: các số đếm theo-từng-loại-thao-tác của khối lượng công việc hỗn hợp được kiểm chứng là giống hệt nhau qua các ngôn ngữ với một hạt giống và một quần thể UE đang hoạt động cho trước, xác nhận rằng cả hai bản hiện thực được điều khiển đi theo đúng cùng các đường mã.

7. Thiết lập thực nghiệm và Đánh giá

7.1 Cấu hình testbed

Các thí nghiệm hiệu năng chạy trên một máy Linux chạy-trực-tiếp-trên-phần-cứng (bare-metal) chuyên dụng với phần cứng giống hệt nhau cho cả hai bản hiện thực, tuân theo một quy trình được ghi tài liệu nhằm loại bỏ các nguồn nhiễu độ trễ đuôi có thể tránh được. Tiến trình đo được ghim vào các lõi vật lý được cô lập bằng `taskset`; bộ điều tần được đặt ở `performance`; turbo/boost và, ở những chỗ đo hành vi theo-từng-lõi, đa luồng đồng thời (SMT) được tắt; và một phần khởi động ban đầu của mỗi lần chạy được loại bỏ sao cho các hiệu ứng cache-lạnh và bộ-cấp-phát-lạnh không làm ô nhiễm trạng thái ổn định. Mỗi điểm quét được lặp lại nhiều lần và báo cáo dưới dạng trung vị với một thước đo độ phân tán chứ không phải một lần chạy đơn, và bốn biến thể ngôn-ngữ×bộ-cấp-phát được đan xen trong mỗi lần lặp sao cho sự trôi nhiệt hay nền ảnh hưởng chúng như nhau. Môi trường (kiểu CPU, nhân, phiên bản trình biên dịch, trạng thái bộ điều tần và turbo) được ghi nhận tự động bên cạnh các kết quả, và toàn bộ cấu hình cùng kịch bản được công bố để tái lập. Công việc chức năng (kiểm thử tương đương, tiêm lỗi, và nguyên mẫu FFI) không nhạy với môi trường và được tiến hành trên một máy phát triển WSL2, nhưng các con số độ trễ và tài nguyên chủ chốt chỉ được tạo trên testbed bare-metal, vì WSL2 không phơi bày các điều khiển tần số và lập lịch mà quy trình đòi hỏi.

7.2 Khối lượng công việc và các phép đo đánh giá

Một khối lượng công việc viễn thông tổng hợp nhưng được biện minh theo đặc tả được sinh bởi một bộ điều khiển tất định, giống nhau về cấu trúc ở cả hai bản hiện thực, sao cho hai bản được vận dụng đi theo các đường mã giống hệt nhau. Hai khối lượng công việc được cung cấp. Khối lượng *signal* nạp trước một quần thể các UE đã kết nối rồi dồn dập vào đường nóng `handle_signaling`, cô lập độ trễ theo-từng-

thông-điệp ở trạng thái ổn định. Khối lượng *mixed* là một bộ sinh vòng đời RAN cố định dẫn mỗi UE đi qua một chuỗi thủ tục thực tế — tạo ngữ cảnh, thiết lập kết nối, báo hiệu, thiết lập mang, cấu hình lại/chuyển giao (begin/complete/abort), và release — dùng một thứ tự rút SplitMix64 cố định và một mô hình trạng-thái-kỳ-vọng theo từng khe sao cho mọi thao tác phát ra đều hợp lệ. Vì thứ tự rút là cố định, các số đếm theo-từng-loại-thao-tác giống hệt nhau trong C++ và Rust với một quần thể UE đang hoạt động và một hạt giống cho trước, điều này đồng thời đóng vai trò tín hiệu tương đương được mô tả trong Mục 6.7. Các tham số điều khiển là tốc độ sự kiện mục tiêu và số UE đang hoạt động, được quét từ tải danh định qua bão hoà và quá tải; một lịch gửi cố định với hiệu chỉnh biểu đồ được dùng sao cho thiên lệch bỏ-sót-phối-hợp được tránh khi hệ thống khựng lại [31]. Ba chiến dịch đo được chạy: một phép quét tải cho hiệu năng, nghiên cứu tiêm lỗi, và một nguyên mẫu di trú FFI.

7.3 Thu thập dữ liệu và các thước đo

Nghiên cứu ghi nhận tính dự đoán được của độ trễ (giá trị trung bình, trung vị, p99/p99.9/p99.99, và độ trễ lớn nhất theo từng sự kiện, cộng với độ dao động), thông lượng (số sự-kiện/giây được duy trì và thông lượng tại một mục tiêu mức dịch vụ (SLO) độ trễ cố định), mức dùng tài nguyên (mức dùng CPU, bộ nhớ theo thời gian, các mẫu cấp phát), độ vững chắc tương tranh, an toàn (các lớp lỗi được ngăn tại biên dịch so với tại thời gian chạy), và các đại lượng thay thế cho chất lượng phần mềm (số dòng mã, độ phức tạp chu trình, và số lượng cùng phạm vi các khối `unsafe` của Rust). Tuân theo hướng dẫn đã được thiết lập về đánh giá hiệu năng chặt chẽ về mặt thống kê, mỗi cấu hình được đo qua nhiều lần lặp lại độc lập, đan xen và báo cáo dưới dạng trung vị với một khoảng tin cậy bootstrap 95% thay vì một con số duy nhất, sao cho tính phi tất định giữa các lần chạy không dẫn đến các kết luận không được ủng hộ [16]. Chỉ dữ liệu tổng hợp được dùng, nên không có dữ liệu cá nhân, thuê bao hay bảo mật nào được xử lý.

7.4 Dữ liệu đánh giá

Tập dữ liệu thu thập được gồm các biểu đồ độ trễ theo từng sự kiện, các chuỗi thời gian thông lượng và mức dùng tài nguyên cho mỗi mức tải và ngôn ngữ, các kết quả theo-từng-lỗi của các lần chạy tiêm lỗi, và các phép đo tương tác của nguyên mẫu FFI. Một chiến dịch sơ bộ đầy đủ (hai ngôn ngữ × hai bộ cấp phát × tám mức tải × bốn lần lặp) đã được thu thập trên môi trường phát triển và được dùng bên dưới

để vận dụng đường ống phân tích; các con số hiệu năng sẽ được làm mới bằng các số đạt chuẩn testbed bằng cách chạy lại cùng các kịch bản trên máy chủ của Mục 7.1.

7.5 Các kết quả đánh giá

Các kết quả hiệu năng và tài nguyên bên dưới là các phép đo sơ bộ (môi trường phát triển); các kết quả an toàn và di trú không phụ thuộc vào vệ sinh testbed và là kết quả cuối cùng.

7.5.1 Hiệu năng (RQ1, RQ3)

Giả thuyết là Rust và C++ đạt được thông lượng trung bình tương tự nhau nhưng có thể khác biệt đo được về độ trễ đuôi và độ dao động khi quá tải, và rằng bất kỳ chi phí phụ trội nào từ mô hình an toàn của Rust cũng là nhỏ trong các khối lượng công việc thực tế. Phần đánh giá báo cáo ở đâu, nếu có, mỗi ngôn ngữ thắng và thắng bao nhiêu, qua phép quét tải lên tới và vượt qua bão hoà.

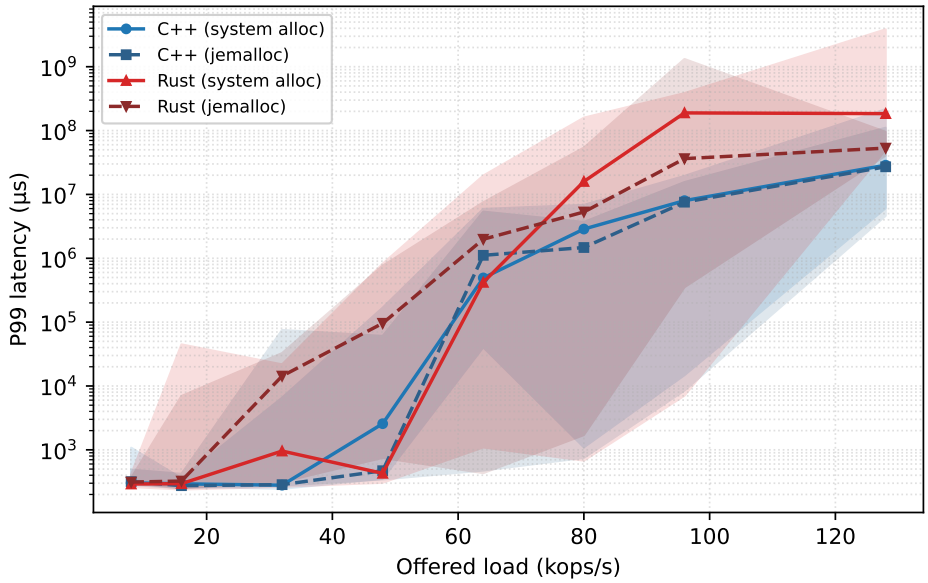
Các hình và bảng trong mục này được sinh bởi một kịch bản tái lập được duy nhất (`bench/plot_figures.py`) trực tiếp từ các nhật ký đo thô, nên chúng có thể được tái tạo y nguyên một khi chiến dịch được chạy lại trên testbed bare-metal của Mục 7.1. Các con số báo cáo ở đây là *sơ bộ*, thu thập trên môi trường phát triển (WSL2), vốn tường minh là không đạt chuẩn testbed cho các tuyên bố về độ trễ đuôi; chúng được đưa vào để vận dụng toàn bộ đường ống đo-và-phân-tích và để minh hoạ loại bằng chứng mà mỗi câu hỏi nghiên cứu sẽ dựa vào. Mỗi điểm tổng hợp bốn lần lặp đan xen, và các dải tô bóng là các khoảng tin cậy bootstrap phi tham số 95% trên trung vị.

Bảng 7.1 tóm tắt phân bố độ trễ theo-từng-thao-tác ở một tải được duy trì 16 kops/s, một điểm vận hành tiêu biểu nằm thoải mái dưới ngưỡng khởi phát quá tải trên máy này. Qua bốn biến thể bản dựng, các trung vị (p50) nhất trí trong vòng vài micro-giây và các giá trị p99 nằm trong khoảng 20% của nhau; các ngôn ngữ chỉ tách nhau ở phần đuôi xa (p99.9 và max), nơi lựa chọn bộ cấp phát quan trọng ngang với ngôn ngữ.

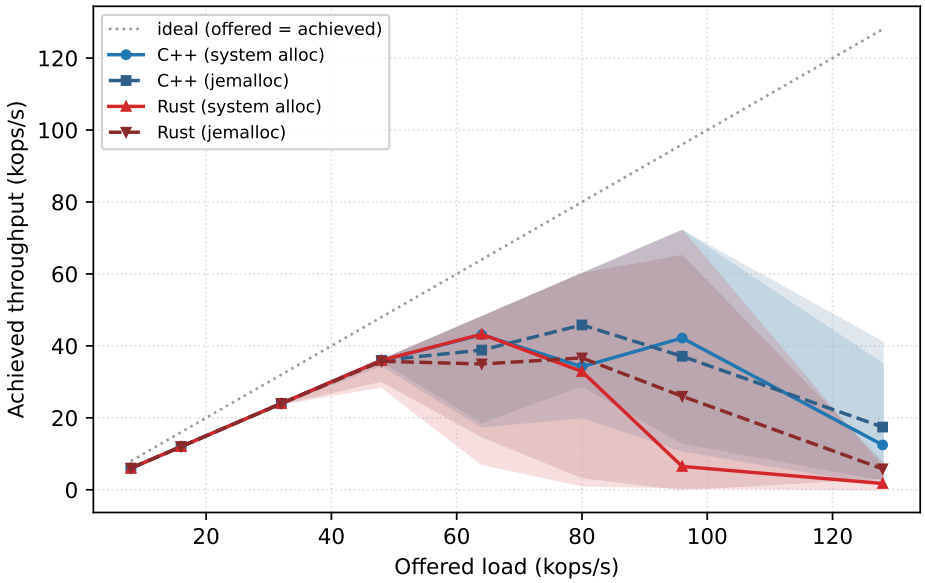
Hình 7.1 vẽ độ trễ p99 theo tải đưa vào. Mọi biến thể bám theo một phần đuôi thấp, bằng phẳng (vài trăm micro-giây) lên tới khoảng 50 kops/s, vượt quá đó thiết kế chặn-nghẽn-bằng-cách-chặn đẩy hệ thống vào quá tải và độ trễ đầu-cuối, vốn bao gồm thời gian chờ trong các hàng đợi có chặn, leo dốc đứng. Sự khởi phát và hình dạng của vách quá tải này, chứ không phải hành vi sâu trong bão hoà (vốn bị chi phối bởi bộ lập lịch WSL2 ở đây), là đại lượng cần quan tâm, và chính là cái mà chiến dịch bare-metal sẽ xác định.

Bảng 7.1: Độ trễ theo-từng-thao-tác ở một tải được duy trì 16 kops/s (khối lượng công việc vòng-đời-RAN hỗn hợp, 8 mảnh). Giá trị tính bằng micro-giây, trung vị qua bốn lần lặp. Các phép đo sơ bộ, môi trường phát triển (WSL2).

Biến thể	trung bình	p50	p90	p99	p99.9	max
C++ (system alloc)	166.3	161.2	206.1	293.5	523.1	1200.1
C++ (jemalloc)	154.7	151.4	192.2	274.7	546.2	1782.5
Rust (system alloc)	151.9	145.0	190.0	295.4	1234.7	3338.2
Rust (jemalloc)	193.7	150.3	223.7	324.4	837.6	1947.1



Hình 7.1: Độ trễ đuôi (p99) theo tải đưa vào cho bốn biến thể bản dựng. Các đường là trung vị qua bốn lần lặp; các dải tô bóng là các khoảng tin cậy bootstrap 95%. Trục y theo thang lô-ga-rit. Dữ liệu sơ bộ (WSL2).



Hình 7.2: Thông lượng đạt được theo tải đưa vào. Đường chấm đánh dấu chế độ lý tưởng đưa-vào-bằng-đạt-được. Dữ liệu sơ bộ (WSL2).

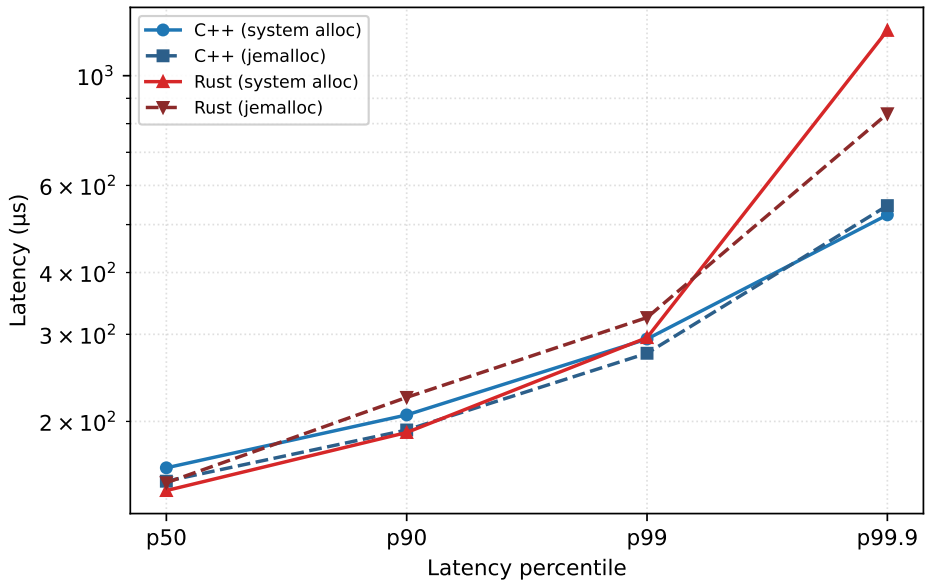
Hình 7.2 cho thấy thông lượng đạt được theo tải đưa vào. Mọi biến thể bám theo đường lý tưởng cho tới khi nó bão hoà gần 40–45 kops/s rồi tụt lại khi các bên sản xuất bị chặn trên các hàng đợi đầy, xác nhận rằng sự phân kỳ trong Hình 7.1 trùng với đầu gối thông lượng chứ không phải với bất kỳ hiệu ứng đặc-thù-ngôn-ngữ nào ở tải thấp.

Hình 7.3 cô lập hình dạng của phân bố ở tải duy trì 16 kops/s bằng cách vẽ độ trễ theo phân vị. Các đường gần như không phân biệt được cho tới p99 và chỉ tách ra ở p99.9, minh hoạ rằng các khác biệt thú vị giữa hai ngôn ngữ nằm hoàn toàn ở phần đuôi, đó chính là lý do vì sao một phương pháp có-nhận-thức-phân-vị (các biểu đồ HDR với hiệu chỉnh bỏ-sót-phối-hợp) được dùng xuyên suốt.

Đối với RQ3, Bảng 7.2 và Hình 7.4 báo cáo mức dùng tài nguyên ở cùng tải duy trì. Kích thước tập thường trú đỉnh là khiêm tốn và tương tự nhau qua các biến thể (từ một chữ số đến hai chữ số thấp MiB); bản dựng C++ dùng-bộ-cấp-phát-hệ-thống là gọn nhất, và jemalloc đánh đổi vài megabyte RSS thêm để lấy các lợi ích về độ trễ đuôi của nó. Mức dùng CPU là tương đương qua cả bốn, nhất quán với thiết kế một-luồng-trên-mỗi-mảnh chung làm cùng một công việc trong mỗi ngôn ngữ.

7.5.2 An toàn qua tiêm lỗi (RQ2)

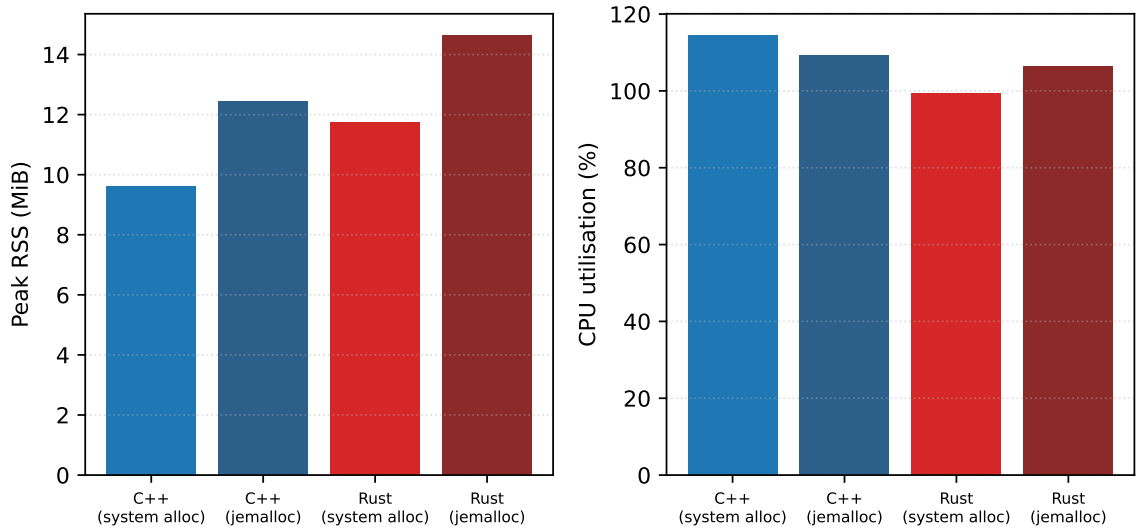
Thay vì tuyên bố rằng Rust không có lỗi, một danh mục gồm năm khiếm khuyết C++ kinh điển được cố ý tiêm vào: sử-dụng-sau-khi-giải-phóng, giải-phóng-hai-lần, vô hiệu hoá iterator, tranh chấp dữ liệu, và biến đổi chia sẻ không được bảo vệ. Mỗi



Hình 7.3: Độ trễ theo phân vị ở tải duy trì 16 kops/s; trục y theo thang lô-ga-rit. Bốn biến thể trùng nhau cho tới p99 và chỉ phân kỳ ở phần đuôi xa. Dữ liệu sơ bộ (WSL2).

Bảng 7.2: Mức dùng tài nguyên ở một tải được duy trì 16 kops/s: thông lượng đạt được, RSS đỉnh, và tổng mức dùng CPU. Trung vị qua bốn lần lặp. Các phép đo sơ bộ, môi trường phát triển (WSL2).

Biến thể	thông lượng (kops/s)	RSS đỉnh (MiB)	CPU (%)
C++ (system alloc)	12.0	9.6	114.3
C++ (jemalloc)	12.0	12.4	109.2
Rust (system alloc)	12.0	11.8	99.2
Rust (jemalloc)	12.0	14.6	106.5



Hình 7.4: Kích thước tập thường trú đỉnh (trái) và mức dùng CPU (phải) theo từng biến thể bản dựng ở 16 kops/s. Dữ liệu sơ bộ (WSL2).

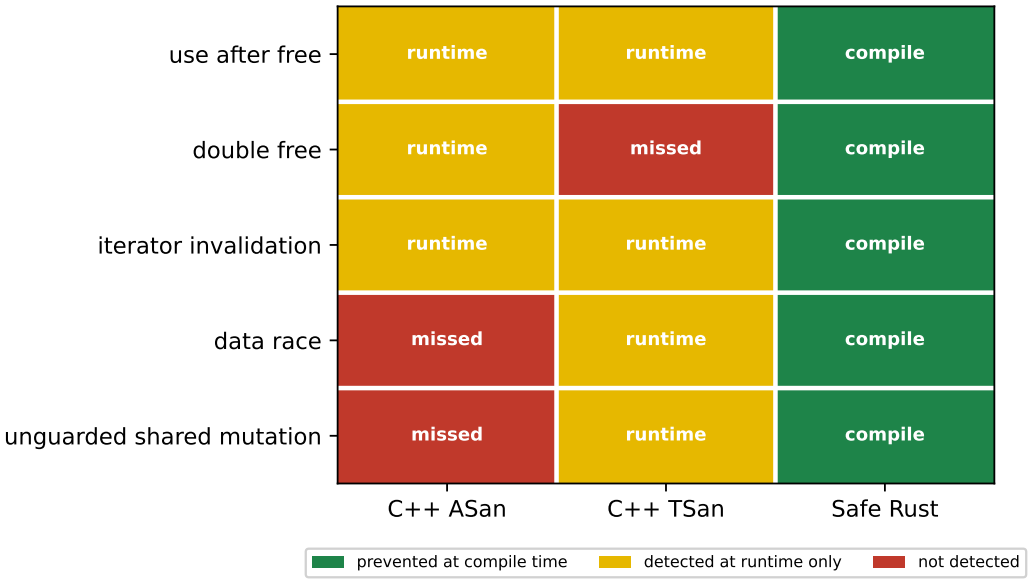
Bảng 7.3: Phân loại lớp lỗi: các kết quả đo được cho danh mục được tiêm. C++ dưới `g++ 13.3` với ASan/TSan; Rust dưới `rustc 1.96`, edition 2021. "Ngăn tại biên dịch" nghĩa là chương trình Rust an toàn không biên dịch được.

Lớp lỗi	ASan	TSan	Kết quả Rust
Sử-dụng-sau-khi-giải-phóng	phát hiện	phát hiện	ngăn tại biên dịch (E0499)
Giải-phóng-hai-lần	phát hiện	không phát hiện	ngăn tại biên dịch (E0382)
Vô hiệu hoá iterator	phát hiện	phát hiện	ngăn tại biên dịch (E0502)
Tranh chấp dữ liệu	không phát hiện	phát hiện	ngăn tại biên dịch (E0373)
Biến đổi chia sẻ không bảo vệ	không phát hiện	phát hiện	ngăn tại biên dịch (E0502)

khiếm khuyết là một chương trình nhỏ, tự chứa, mang màu sắc lĩnh vực của phân hệ (một con trỏ cũ trỏ vào bản đồ ngữ cảnh, giải phóng một ngữ cảnh hai lần, biến đổi một danh sách mang trong khi lặp qua nó, hai worker cập nhật một trường ngữ cảnh mà không đồng bộ hoá), sao cho nó ánh xạ lên một lỗi hợp lý trong dịch vụ Quản lý Ngữ cảnh UE. Với mỗi khiếm khuyết C++, nghiên cứu dựng và chạy chương trình dưới cả AddressSanitizer (ASan) và ThreadSanitizer (TSan) và ghi nhận công cụ nào, nếu có, phát hiện nó; với mỗi khiếm khuyết, cấu trúc tương tự được viết trong Rust an toàn và biên dịch, và kết quả của trình biên dịch được ghi nhận. Danh mục, bộ chạy, và các nhật ký công cụ thô được công bố cùng luận văn.

Các kết quả được cho trong Bảng 7.3, đo với `g++ 13.3` (`-std=c++23`, `-O0` để trình tối ưu không thể loại bỏ một khiếm khuyết được tiêm) và `rustc 1.96` (edition 2021). Phát hiện trung tâm là rõ ràng: *mọi* khiếm khuyết được tiêm đều là một lỗi biên dịch trong Rust an toàn, nên không lỗi nào có thể viết ra được, trong khi ở C++ mỗi lỗi là mã hợp lệ mà khiếm khuyết chỉ biểu hiện tại thời gian chạy, và chỉ khi một kiểm thử thực sự thực thi đường có lỗi. Các khiếm khuyết an toàn bộ nhớ bị ASan bắt; hai khiếm khuyết tương tranh chỉ bị TSan bắt và vô hình với ASan, nhấn mạnh rằng không một bộ dò lỗi C++ đơn lẻ nào phủ toàn bộ danh mục. Việc phát hiện tại thời gian chạy còn đòi hỏi dựng một tệp nhị phân được đo lường riêng và trả chi phí thời gian và bộ nhớ của bộ dò lỗi, trong khi các lỗi Rust được báo cáo trên mọi bản dựng thông thường bất kể độ phủ kiểm thử. Các mã trong ngoặc ở cột cuối là các chẩn đoán `rustc` kích hoạt (ví dụ `E0499`, các mượn khả biến xung đột; `E0502`, một mượn khả biến trong khi một mượn chia sẻ đang sống; `E0382`, dùng một giá trị đã bị di chuyển; `E0373`, một closure sống lâu hơn dữ liệu được mượn khi nó thoát sang một luồng khác).

Hình 7.5 trình bày cùng các kết quả dưới dạng một ma trận được mã hoá màu, được sinh từ nhật ký máy-đọc-được của bộ chạy (`faults/results/taxonomy.csv`). Nó làm cho sự bất đối xứng trung tâm trở nên hữu hình trong nháy mắt: toàn bộ



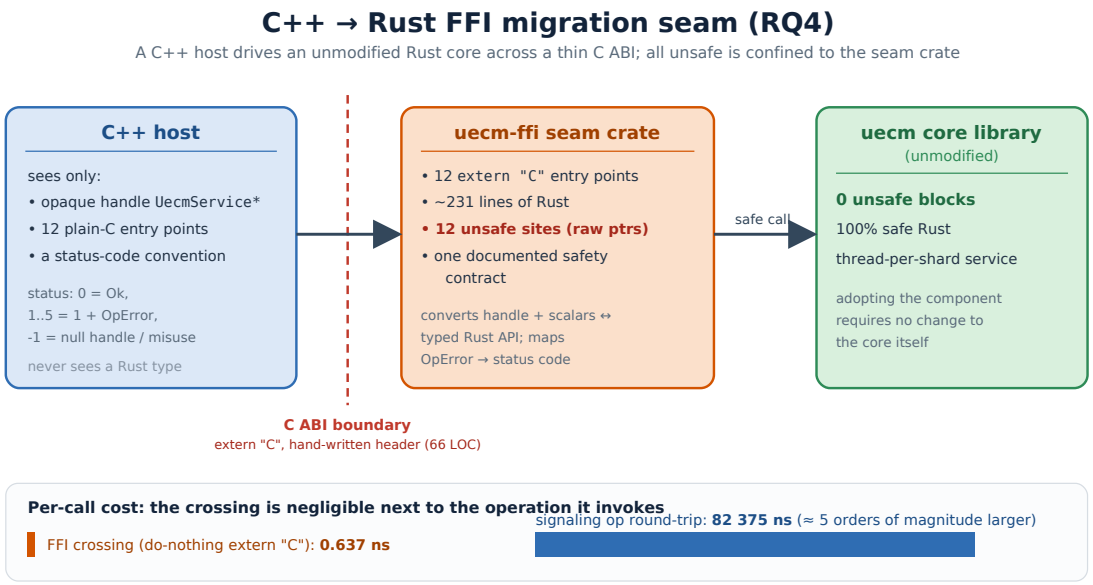
Hình 7.5: Ma trận kết quả lớp lỗi. Các ô xanh là các khiếm khuyết được ngăn tại thời điểm biên dịch (Rust an toàn); các ô hổ phách là các khiếm khuyết biên dịch được trong C++ nhưng bị bắt tại thời gian chạy bởi bộ dò lỗi cho trước; các ô đỏ là các khiếm khuyết mà bộ dò lỗi không phát hiện. Được sinh từ các nhật ký của bộ chạy tiêm lỗi.

cột bên phải là màu xanh (mọi khiếm khuyết đều được ngăn tại thời điểm biên dịch trong Rust an toàn), trong khi các cột C++ là một mảnh chắp vá của các lần phát hiện tại thời gian chạy và các lần bỏ sót hoàn toàn, vì không một bộ dò lỗi đơn lẻ nào phủ toàn bộ danh mục.

7.5.3 Di trú (RQ4)

Việc áp dụng tăng dần được đánh giá bằng một nguyên mẫu hoạt động được, mô hình hoá hướng di trú thực tế: một host C++ sẵn có giữ một thành phần Rust sau một C ABI. Dịch vụ Rust được phơi bày qua một đường ghép `extern "C"` mỏng trong một crate *riêng* phụ thuộc vào thư viện lõi không đổi, nên việc áp dụng thành phần không đòi hỏi thay đổi nào ở bản thân thành phần. Host C++ chỉ thấy một handle mờ và một tập các điểm vào C thuần (một header viết tay, không có ràng buộc được sinh tự động), và điều khiển toàn bộ vòng đời UE qua ranh giới; mô hình lỗi được truyền tải trung thực (ví dụ, một lần thiết lập kết nối trùng lặp trả về `InvalidState` qua đường ghép).

Nguyên mẫu cho ra các con số chi-phí-tương-tác cụ thể, được tóm tắt trong Bảng 7.4. Đường ghép nhỏ, mười hai điểm vào C-ABI trong khoảng 230 dòng Rust cộng một header C 66 dòng, không có phụ thuộc FFI bên thứ ba. Ranh giới an toàn sạch sẽ: thư viện lõi vẫn không có `unsafe`, và việc xử lý con-trỏ-thô mà C ABI đòi hỏi được giới hạn trong crate đường ghép (mười hai vị trí `unsafe` của nó) dưới một

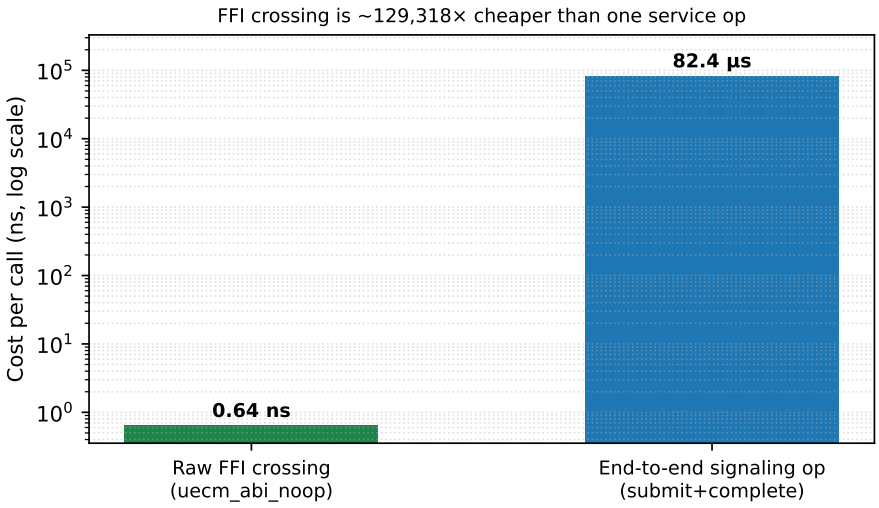


Hình 7.6: Đường ghép di trú FFI C++-sang-Rust. Host C++ chỉ thấy một handle mờ, mười hai điểm vào C thuần, và một quy ước mã trạng thái; một crate đường ghép `extern "C"` mỏng tập trung toàn bộ `unsafe` con-trỏ-thô (mười hai vị trí) và gọi lõi Rust không đổi, vốn vẫn không có `unsafe`. Việc băng qua ranh giới không-làm-gì tốn một phần nhỏ của một nano-giây, thấp hơn khoảng năm bậc độ lớn so với chi phí của một thao tác báo hiệu.

hợp đồng an toàn được ghi tài liệu duy nhất, nên phía C++ không bao giờ thấy một kiểu Rust. Điều then chốt là, ranh giới không áp đặt thuế mỗi-lời-gọi đo được nào: như Hình 7.7 cho thấy, một lần băng qua không-làm-gì đo được thấp hơn nhiều một nano-giây mỗi lời gọi (một lời gọi gián tiếp thông thường), thấp hơn khoảng năm bậc độ lớn so với chi phí đầu-cuối của một thao tác báo hiệu, vốn bị chi phối bởi vòng khứ hồi gửi-và-hoàn-tất xuyên-luồng của dịch vụ mà một bên gọi bản-địa (không-FFI) cũng trả y hệt. Do đó công sức kỹ nghệ chính nằm ở việc *thiết kế* một bề mặt API tương thích C — một handle mờ, các thao tác định kiểu vô hướng, và một quy ước trạng thái — chứ không phải ở bất kỳ chi phí phụ trội thời gian chạy nào. Hướng ngược lại (Rust gọi vào C++) là đối xứng nhưng đòi hỏi một lớp bọc C-ABI ở phía C++ và tái du nhập `unsafe` ở bên gọi Rust. Nguyên mẫu và các phép đo của nó được công bố cùng luận văn, và cung cấp thông tin cho các hướng dẫn di trú của Chương 8.

Bảng 7.4: Nguyên mẫu di trú FFI: kích thước đường ghép, ranh giới an toàn, và chi phí tương tác mỗi lời gọi (host C++ gọi thành phần Rust qua một C ABI, `-O2`). Thư viện lõi không đổi và không có `unsafe`; mọi tương tác được giới hạn trong crate đường ghép.

Thước đo	Giá trị
C-ABI entry points	12
Seam size (Rust `lib.rs`)	231 LOC
Hand-written C header	66 LOC
`unsafe` in core library	0
`unsafe` in FFI seam	12
Raw FFI crossing	0.64 ns/call
End-to-end signaling op	82.4 µs/call



Hình 7.7: Chi phí mỗi lời gọi của việc băng qua C ABI (một lời gọi `extern "C"` không-làm-gì) so với chi phí đầu-cuối của một thao tác báo hiệu qua đường ghép. Trục y theo thang lô-ga-rit: việc băng qua FFI là không đáng kể bên cạnh vòng khứ hồi xuyên-luồng của dịch vụ.

8. Thảo luận

8.1 Tóm tắt các phát hiện

Bảng 8.1 thu thập phát hiện chủ chốt cho mỗi câu hỏi nghiên cứu cùng với độ mạnh của bằng chứng hiện tại. Các kết quả an toàn và di trú (RQ2, RQ4) là cuối cùng, vì chúng không phụ thuộc vào môi trường đo; các kết quả hiệu năng và tài nguyên (RQ1, RQ3) được báo cáo từ chiến dịch môi trường phát triển và được kỳ vọng sẽ đúng trên testbed bare-metal, nhưng được gắn nhãn sơ bộ cho tới khi chiến dịch đó được chạy.

8.2 Thảo luận về các câu hỏi nghiên cứu

RQ1 (tính dự đoán được). Phép quét sơ bộ ủng hộ giả thuyết rằng hai ngôn ngữ, đối với khối lượng công việc này, tương đương về hiệu năng trong vùng quan trọng về mặt vận hành: dưới bão hoà, độ trễ trung vị và p99 của chúng không phân biệt được trong phạm vi nhiễu giữa các lần chạy, và phần đuôi chỉ tách ra một khi thiết kế chặn-nghẽn-bằng-cách-chặn đẩy hệ thống vào quá tải. Điều này nhất quán với bằng chứng rộng hơn rằng các kiểm tra an toàn của Rust phần lớn được các CPU hiện đại hấp thụ [13, 8], và nó dịch chuyển câu hỏi thực tiễn từ "Rust có đủ nhanh không?" sang "mỗi ngôn ngữ hành xử thế nào ở vách quá tải?", một câu hỏi mà chiến dịch bare-metal được thiết kế để trả lời với các điều khiển tần số và lập lịch mà WSL2 không thể cung cấp.

RQ2 (an toàn). Kết quả là rõ ràng và không phụ thuộc vào môi trường: trình kiểm tra mượn từ chối cả năm lớp lỗi tại thời điểm biên dịch, nên chúng không thể viết ra được trong Rust an toàn, trong khi ở C++ mỗi lỗi là mã hợp lệ chỉ bị phát hiện tại thời gian chạy và chỉ dưới đúng bộ dò lỗi và một kiểm thử thực thi đường có lỗi. Đây là một khác biệt định tính, loại-bảo-đảm chứ không phải định lượng, và nó ăn khớp với bằng chứng CVE rằng các lỗi an toàn bộ nhớ của Rust luôn liên quan đến `unsafe` [33] và với lời cảnh báo rằng tính không an toàn vẫn có thể lan truyền

Bảng 8.1: Tóm tắt các phát hiện theo từng câu hỏi nghiên cứu và độ mạnh hiện tại của bằng chứng.

RQ	Phát hiện chủ chốt	Trạng thái bằng chứng
RQ1	Rust và C++ bám sát nhau đến bão hoà (trung vị chênh nhau vài micro-giây, p99 trong khoảng ∼20%); khác biệt chỉ xuất hiện ở phần đuôi xa, nơi bộ cấp phát quan trọng ngang với ngôn ngữ.	Sơ bộ (WSL2)
RQ2	Mỗi một trong năm lớp lỗi C++ được tiêm đều được ngăn tại thời điểm biên dịch trong Rust an toàn; trong C++ mỗi lỗi biên dịch được và chỉ bị bắt, nếu có, tại thời gian chạy, và không một bộ dò lỗi đơn lẻ nào phủ toàn bộ danh mục.	Cuối cùng
RQ3	RSS đỉnh và mức dùng CPU tương đương qua các ngôn ngữ; `jemalloc` đánh đổi vài MiB RSS để lấy một phần đuôi ngắn hơn.	Sơ bộ (WSL2)
RQ4	Đường ghép FFI nhỏ, giữ cho lõi không có `unsafe`, và không áp đặt thuế mỗi-lời-gọi đo được nào (băng qua dưới nano-giây so với một thao tác ∼80 μs); chi phí thực sự là thiết kế API, không phải phụ trội thời gian chạy.	Cuối cùng

qua các phụ thuộc [14], đó là lý do vì sao việc giữ cho lõi không có `unsafe` (như đã làm ở đây) là quan trọng.

RQ3 (chi phí phụ trội). Không quan sát thấy hình phạt bộ nhớ hay CPU có hệ thống nào cho mô hình an toàn của Rust ở các mức tải được kiểm tra; mức dùng tài nguyên bị chi phối bởi thiết kế chung chứ không phải bởi ngôn ngữ, và lựa chọn bộ cấp phát là một đòn bẩy lớn hơn lựa chọn ngôn ngữ. Cũng như với RQ1, các con số tuyệt đối chờ testbed, nhưng bức tranh *tương đối* (không có hình phạt ở mức ngôn ngữ) được kỳ vọng là vững chắc.

RQ4 (di trú). Nguyên mẫu cho thấy việc áp dụng tăng dần là rẻ về mặt kỹ thuật: một host C++ có thể gọi một lõi Rust không đổi qua một đường ghép C-ABI mỏng với chi phí mỗi-lời-gọi dưới nano-giây, giới hạn toàn bộ `unsafe` vào đường ghép. Do đó chi phí chi phối là của con người, không phải của máy: thiết kế một bề mặt tương thích C và, theo các nghiên cứu áp dụng [15], leo lên đường cong học tập của Rust. Điều này định khung lại rủi ro di trú như một mối quan tâm về tổ chức chứ không phải về hiệu năng.

8.3 Công trình liên quan

Các phép so sánh Rust-đối-với-C/C++ trước đây tập trung vào hiệu năng đa dụng và năng lượng [26], và các khảo sát đo hiệu năng đa ngôn ngữ báo cáo rằng Rust sánh với C và C++ về hiệu năng trong khi bổ sung an toàn [10]. Công trình hình thức và thực nghiệm thiết lập tính đúng đắn của phân đoạn an toàn của Rust và việc dùng `unsafe` có kỷ luật: RustBelt cung cấp các nền tảng được máy-kiểm-chứng cho các tuyên bố về an toàn [18, 19] và mô hình Stacked Borrows hình thức hoá các quy tắc bí danh mà mã `unsafe` phải tôn trọng [17], trong khi các nghiên cứu thực nghiệm về mã thực và về tất cả các CVE an toàn bộ nhớ đã biết cho thấy `unsafe` được dùng tiết chế và rằng các khiếm khuyết an toàn bộ nhớ luôn truy về được nó [7, 28, 33]; một nghiên cứu bổ sung cảnh báo rằng dù vậy tính không an toàn vẫn lan truyền qua các chuỗi lời gọi phụ thuộc, nên hầu hết các thư viện không thể được kiểm tra tĩnh đầy đủ [14]. Các báo cáo của công nghiệp và chính phủ tạo động lực cho an toàn bộ nhớ từ góc độ bảo mật và nay kêu gọi di trú sang các ngôn ngữ an toàn bộ nhớ [22, 32, 11, 25, 29]; sự khó khăn dai dẳng trong việc kiềm chế các cuộc tấn công hỏng-bộ-nhớ trong C/C++ chỉ bằng các biện pháp giảm thiểu thời gian chạy là phông nền bảo mật cho sự chuyển dịch này [30]. Về phía hiệu năng, các nghiên cứu hướng-hệ-thống lập luận rằng Rust cung cấp an toàn với chi phí thời gian chạy về cơ bản bằng không và khả thi trong các lĩnh vực mức-thấp khắt khe: Rust cưỡng chế

an toàn mà không có chi phí phụ trội thu gom rác và cho phép các kỹ thuật hiệu quả bất khả thi trong C [8], các hệ thống nhúng/thời-gian-thực như một hypervisor Rust đã được cho thấy đạt được an toàn bộ nhớ mà không hy sinh hiệu năng [23], hạ tầng đám mây sản xuất như AWS Firecracker được xây dựng bằng Rust [6], và trong lĩnh vực mạng một trình điều khiển không-gian-người-dùng viết bằng Rust theo lối thành ngữ chạy trong vòng vài phần trăm so với bản C tham chiếu của nó [13]. Góc độ tính dự đoán được dựa trên tài liệu về độ trễ đuôi và độ trễ mức micro-giây [12, 9, 21], bối cảnh RAN dựa trên xu hướng phân rã O-RAN [27], và góc độ áp dụng dựa trên các nghiên cứu định tính báo cáo rằng rào cản chính đối với Rust là đường cong học tập của nó chứ không phải chi phí thời gian chạy [15]. Phương pháp đo tuân theo hướng dẫn đã được thiết lập về đánh giá hiệu năng chặt chẽ về mặt thống kê, dựa-trên-lặp-lại [16], và nghiên cứu tiêm lỗi tuân theo thực tiễn tiêm-lỗi-phần-mềm chuẩn [24]. Luận văn này bổ trợ cho khối công trình đó bằng cách nhắm vào tính dự đoán được của độ trễ đuôi và chi phí di trú cho một khối lượng công việc mặt điều khiển viễn thông khi quá tải — một lĩnh vực ứng dụng và chế độ vận hành mà các phép so sánh trước đây không bao phủ.

Không phải tất cả bằng chứng này đều chỉ về cùng một hướng, và luận văn xem xét các phát hiện có tính dè dặt một cách nghiêm túc thay vì chỉ trích dẫn những phát hiện thuận lợi. Ngay cả kết quả mạng tích cực cũng ghi nhận một chi phí thực: trình điều khiển Rust theo lối thành ngữ phát ra nhiều lệnh hơn đáng kể trên mỗi gói so với bản C tham chiếu của nó và chậm hơn một cách đo được về mặt tuyệt đối, các kiểm tra an toàn của nó được thực-thi-ngoài-thứ-tự hấp thụ chứ không phải thực sự miễn phí [13]; do đó cách định khung "không-chi-phí" [8] nên được đọc là phụ-thuộc-khối-lượng-công-việc hơn là phổ quát, đó chính là lý do vì sao hành vi phần đuôi phải được đo trên khối lượng công việc mục tiêu chứ không phải được giả định. Câu chuyện an toàn cũng không tuyệt đối: các nghiên cứu thực nghiệm về Rust thực tế phát hiện rằng các lỗi an toàn bộ nhớ và an toàn luồng vẫn xảy ra trong thực tế, trong đó có các lỗi chặn (blocking bug) và đồng bộ hoá bị dùng sai [28], và rằng một tỉ lệ đáng kể của hệ sinh thái không thể được kiểm tra tĩnh đầy đủ vì `unsafe` lan truyền qua các chuỗi phụ thuộc [14]. Các điểm dè dặt này không lật ngược lý lẽ cho Rust, nhưng chúng tạo động lực cho hai lựa chọn thiết kế của luận văn: đo tính dự đoán được và việc ngăn chặn lỗi trực tiếp trên phân hệ thay vì nhập khẩu các kết luận đa dụng, trường hợp trung bình, và giữ cho lõi nhạy-an-toàn không có `unsafe` sao cho điểm dè dặt lan-truyền-qua-phụ-thuộc không áp dụng cho nó.

8.4 Các mối đe doạ đối với tính hợp lệ và biện pháp giảm thiểu

Mối đe doạ trung tâm là thiên lệch hiện thực giữa hai bản; nó được giảm thiểu bằng đặc tả chung đã đóng băng (Mục 6.1), các thuật toán, cấu trúc dữ liệu, và mô hình tương tranh giống hệt nhau, bộ cấp phát và tối ưu hoá bản dựng giống hệt nhau, và kiểm thử tương đương chức năng độc lập (Mục 6.7). Độ tin cậy của các kết quả định lượng dựa trên các lần chạy lặp lại với các khoảng tin cậy được báo cáo và việc ghi nhận độ trễ dựa-trên-biểu-đồ tránh thiên lệch bỏ-sót-phối-hợp [31]. Các mối đe doạ còn lại gồm sự bất đối xứng kỹ năng của một tác giả duy nhất, việc dùng một khối lượng công việc tổng hợp thay vì các vết sản xuất được ghi lại, một phân hệ đơn giản hoá duy nhất (giới hạn khả năng khái quát hoá sang PHY/L2 hay phần cứng khác), và câu hỏi liệu các thước đo được chọn có nắm bắt trung thực tính dự đoán được và tính dễ-mắc-lỗi hay không; các phát hiện định tính được báo cáo đúng như bản chất của chúng.

9. Kết luận và Công việc tương lai

Luận văn này so sánh C++ hiện đại và Rust trên một dịch vụ Quản lý Ngữ cảnh UE được mô hình hoá theo một thành phần gNB-CU-CP thực tế của Nokia, dọc theo bốn chiều: tính dự đoán được của độ trễ đuôi, việc ngăn chặn lỗi tại thời điểm biên dịch, chi phí phụ trội của mô hình an toàn, và chi phí áp dụng tăng dần. Được xây dựng dựa trên một đặc tả đã đóng băng duy nhất và được điều khiển đi qua các đường mã giống hệt nhau, hai bản hiện thực cho phép cô lập các ảnh hưởng của ngôn ngữ khỏi các khác biệt về thiết kế và kỹ năng.

Ba kết luận rút ra từ bằng chứng. Thứ nhất, về an toàn, khác biệt là về *bản chất*: Rust an toàn từ chối cả năm lớp lỗi được tiêm tại thời điểm biên dịch, biến một lớp các hiểm hoạ thời gian chạy mà C++ chỉ có thể hy vọng bắt được bằng các kiểm thử được đo-lường-bằng-bộ-dò-lỗi thành các lỗi hoàn toàn không thể viết ra được. Thứ hai, về di trú, chi phí bị chi phối bởi thiết kế API và đường cong học tập chứ không phải bởi bất kỳ thuế thời gian chạy nào: một lõi Rust không đổi có thể được điều khiển từ C++ qua một đường ghép mỏng, cô-lập-`unsafe` với chi phí mỗi-lời-gọi dưới nano-giây. Thứ ba, về hiệu năng, bằng chứng sơ bộ cho thấy hai ngôn ngữ tương đương trong vùng liên quan về mặt vận hành nằm dưới bão hoà, với các khác biệt chỉ giới hạn ở phần đuôi xa; xác nhận điều này trên một testbed bare-metal là bước còn lại quan trọng nhất, và các kịch bản được công bố tái tạo mọi hình và bảng một khi dữ liệu đó được thu thập.

Một số hướng mở rộng công trình: phát lại các vết sản xuất (đã ẩn danh) được ghi lại thay vì một khối lượng công việc tổng hợp; mở rộng phép so sánh sang các phân hệ mặt điều khiển bổ sung hoặc sang đường L2/PHY nặng-dữ-liệu; đánh giá các runtime bất đồng bộ (async) của Rust so với mô hình một-luồng-trên-mỗi-mảnh được dùng ở đây; và một nghiên cứu tình huống di trú dài hạn hơn theo dõi mật độ lỗi và công sức rà soát khi Rust được áp dụng tăng dần trong một mã nguồn RAN sản xuất.

Tài liệu tham khảo

[1] 3GPP. NG-RAN; architecture description. Technical Specification (TS) 38.401, 3rd Generation Partnership Project (3GPP), 2022. Version 17.x.

[2] 3GPP. NG-RAN; E1 application protocol (E1AP). Technical Specification (TS) 38.463, 3rd Generation Partnership Project (3GPP), 2022. Version 16.x.

[3] 3GPP. NR; NR and NG-RAN overall description; stage 2. Technical Specification (TS) 38.300, 3rd Generation Partnership Project (3GPP), 2022. Version 17.x.

[4] 3GPP. NR; radio resource control (RRC); protocol specification. Technical Specification (TS) 38.331, 3rd Generation Partnership Project (3GPP), 2022. Version 17.x.

[5] 3GPP. Service requirements for the 5G system; stage 1. Technical Specification (TS) 22.261, 3rd Generation Partnership Project (3GPP), 2022. Defines URLLC latency and reliability KPIs.

[6] A. Agache, M. Brooker, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa. Firecracker: Lightweight virtualization for serverless applications. In *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 419–434. USENIX Association, 2020.

[7] V. Astrauskas, C. Matheja, F. Poli, P. Müller, and A. J. Summers. How do programmers use unsafe Rust? *Proceedings of the ACM on Programming Languages*, 4(OOPSLA):136:1–136:27, 2020.

[8] A. Balasubramanian, M. S. Baranowski, A. Burtsev, A. Panda, Z. Rakamarić, and L. Ryzhyk. System programming in Rust: Beyond safety. In *Proceedings of the 16th Workshop on Hot Topics in Operating Systems (HotOS)*, pages 156–161, 2017.

[9] L. Barroso, M. Marty, D. Patterson, and P. Ranganathan. Attack of the killer microseconds. *Communications of the ACM*, 60(4):48–54, 2017.

[10] W. Bugden and A. Alahmar. Rust: The programming language for safety and performance. *arXiv preprint arXiv:2206.05503*, 2022.

[11] Cybersecurity and Infrastructure Security Agency and National Security Agency and Federal Bureau of Investigation and others. The case for memory safe roadmaps: Why both C-suite executives and technical experts need to take memory safe coding seriously. Cybersecurity information sheet, CISA, NSA, FBI, ASD's ACSC, CCCS, NCSC-UK, NCSC-NZ, CERT NZ, Dec. 2023.

[12] J. Dean and L. A. Barroso. The tail at scale. *Communications of the ACM*, 56(2):74–80, 2013.

[13] P. Emmerich, S. Ellmann, F. Bonk, A. Egger, E. García Sánchez-Torija, T. Günzel, S. Di Luzio, A. Obada, M. Stadlmeier, S. Voit, and G. Carle. The case for writing network drivers in high-level programming languages. In *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*, 2019.

[14] A. N. Evans, B. Campbell, and M. L. Soffa. Is Rust used safely by software developers? In *Proceedings of the 42nd International Conference on Software Engineering (ICSE)*, pages 246–257, 2020.

[15] K. R. Fulton, A. Chan, D. Votipka, M. Hicks, and M. L. Mazurek. Benefits and drawbacks of adopting a secure programming language: Rust as a case study. In *Proceedings of the 17th Symposium on Usable Privacy and Security (SOUPS)*, pages 597–616. USENIX Association, 2021.

[16] A. Georges, D. Buytaert, and L. Eeckhout. Statistically rigorous Java performance evaluation. In *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*, pages 57–76, 2007.

[17] R. Jung, H.-H. Dang, J. Kang, and D. Dreyer. Stacked borrows: An aliasing model for Rust. *Proceedings of the ACM on Programming Languages*, 4(POPL):41:1–41:32, 2020.

[18] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. RustBelt: Securing the foundations of the Rust programming language. *Proceedings of the ACM on Programming Languages*, 2(POPL):66:1–66:34, 2018.

[19] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. Safe systems programming in Rust. *Communications of the ACM*, 64(4):144–152, 2021.

[20] S. Klabnik and C. Nichols. *The Rust Programming Language*. No Starch Press, San Francisco, CA, 2nd edition, 2023.

[21] J. Li, N. K. Sharma, D. R. K. Ports, and S. D. Gribble. Tales of the tail: Hardware, OS, and application-level sources of tail latency. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*, pages 9:1–9:14, 2014.

[22] M. Miller. Trends, challenges, and strategic shifts in the software vulnerability mitigation landscape. BlueHat IL, Microsoft Security Response Center (MSRC), 2019. Approximately 70% of Microsoft CVEs are memory-safety issues.

[23] C. Mo, L. Wang, S. Li, K. Hu, and B. Jiang. Rust-Shyper: A reliable embedded hypervisor supporting VM migration and hypervisor live-update. *Journal of Systems Architecture*, 142:102948, 2023.

[24] R. Natella, D. Cotroneo, and H. S. Madeira. Assessing dependability with software fault injection: A survey. *ACM Computing Surveys*, 48(3):44:1–44:55, 2016.

[25] Office of the National Cyber Director. Back to the building blocks: A path toward secure and measurable software. Technical report, The White House, Office of the National Cyber Director (ONCD), Washington, DC, Feb. 2024.

[26] R. Pereira, M. Couto, F. Ribeiro, R. Rua, J. Cunha, J. P. Fernandes, and J. Saraiva. Energy efficiency across programming languages: How do energy, time, and memory relate? In *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering (SLE)*, pages 256–267, 2017.

[27] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia. Understanding O-RAN: Architecture, interfaces, algorithms, security, and research challenges. *IEEE Communications Surveys & Tutorials*, 25(2):1376–1411, 2023.

[28] B. Qin, Y. Chen, Z. Yu, L. Song, and Y. Zhang. Understanding memory and thread safety practices and issues in real-world Rust programs. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 763–779, 2020.

[29] A. Rebert and C. Kern. Secure by design: Google's perspective on memory safety. Google Security Engineering, Technical Report, 2024.

[30] L. Szekeres, M. Payer, T. Wei, and D. Song. SoK: Eternal war in memory. In *Proceedings of the 2013 IEEE Symposium on Security and Privacy (S&P)*, pages 48–62, 2013.

[31] G. Tene. HdrHistogram: A high dynamic range histogram, and the coordinated omission problem. http://hdrhistogram.org/, 2015. Accessed 2026.

[32] J. Vander Stoep and S. Hines. Memory safe languages in Android 13. Google Security Blog, 2022.

[33] H. Xu, Z. Chen, M. Sun, Y. Zhou, and M. R. Lyu. Memory-safety challenge considered solved? an in-depth study with all Rust CVEs. *ACM Transactions on Software Engineering and Methodology*, 31(1):3:1–3:25, 2021.

Phụ lục A. Ví dụ tiêm lỗi

Để minh hoạ cho nghiên cứu tiêm lỗi (Chương 7, RQ2), lỗi sử-dụng-sau-khi-giải-phóng được cho thấy trong cả hai ngôn ngữ. Trong C++, bí danh vào bản đồ ngữ cảnh sống sót qua thao tác `erase` giải phóng nó, nên chương trình sau biên dịch sạch sẽ và chỉ bị AddressSanitizer gắn cờ tại thời gian chạy:

```
std::unordered_map<unsigned long, UeContext> ctxs;
ctxs[1001] = UeContext{1001, 0};
UeContext* p = &ctxs.at(1001);  // alias into the container
ctxs.erase(1001);               // frees the node that p refers to
p->messages_handled += 1;       // use-after-free
```

Chương trình Rust an toàn tương tự không biên dịch được: trình kiểm tra mượn cấm biến đổi bản đồ (`remove`) trong khi tham chiếu `p` vào nó vẫn còn sống, nên lỗi ngay từ đầu không thể viết ra được (lỗi `E0499`):

```
let mut ctxs: HashMap<u64, UeContext> = HashMap::new();
ctxs.insert(1001, UeContext { messages_handled: 0 });
let p = ctxs.get_mut(&1001).unwrap(); // mutable borrow starts
ctxs.remove(&1001);                   // borrow while p live -> rejected
p.messages_handled += 1;              // p used after the remove
```